

Introduction to Parallel Computing

a.y. 2026 – 2027

Prof. Flavio Vella

Lecture 1

Course Material and Information



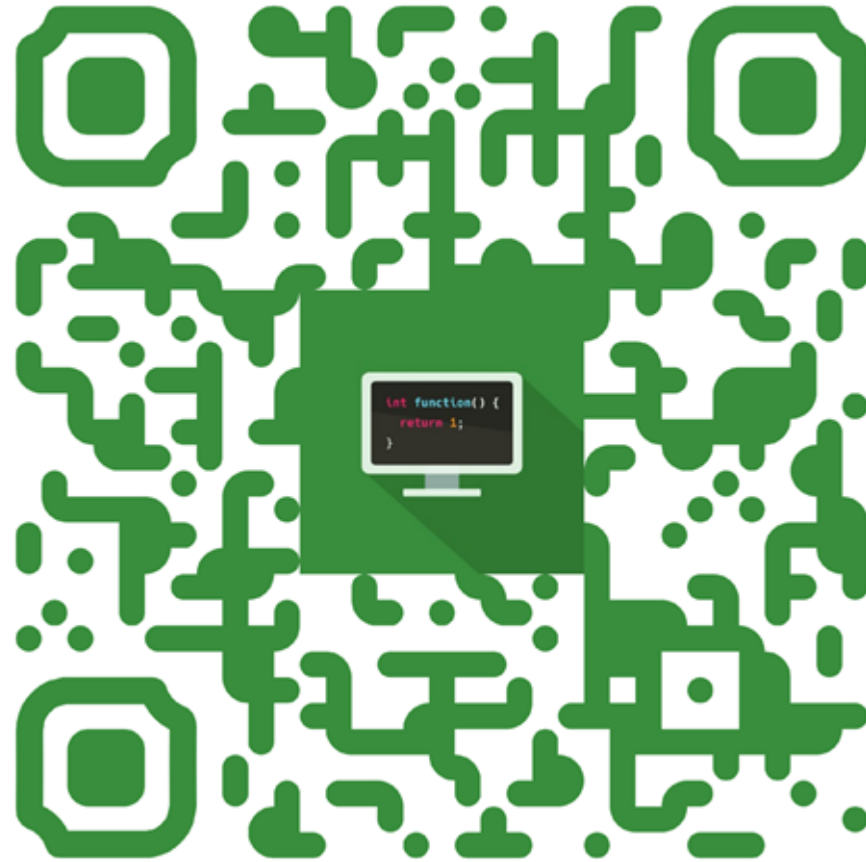
This also applies for
Fundamentals of parallel
programming [146310]

<https://didatticaonline.unitn.it/dol/course/view.php?id=44064>

Telegram



UNITRENTO



<https://t.me/+Yv54lh0mEi4yODI0>



UNITRENTO

Lectures and instructors

Lectures

Prof. Flavio Vella

Teaching Assistant

Dr. Brahmaiah Gandham, Lorenzo Pichetti, Thomas Pasquali and Jacopo Raffi

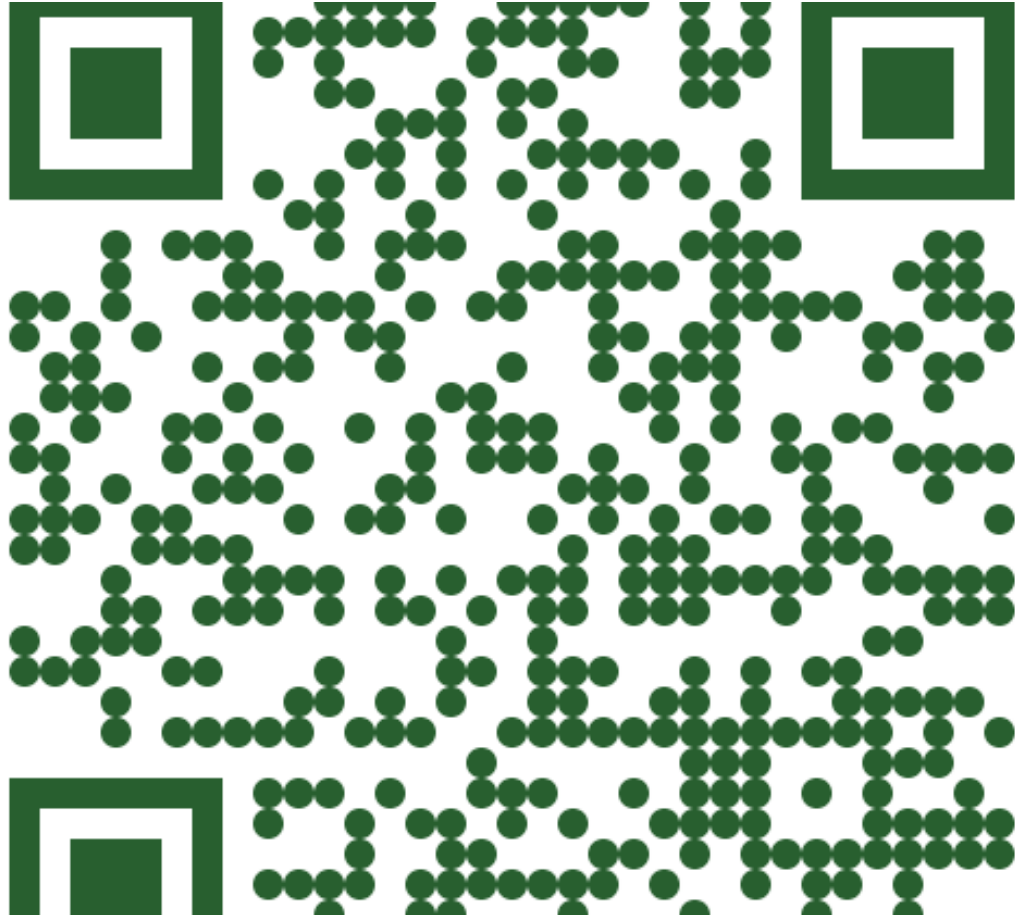
Contact: name.surname@unitn.it (e.g., flavio.vella@unitn.it)

Office hours:

Monday: 11:00 – 12:00

Tuesday: 14:00 – 16:00

Booking page



- You can schedule an appointment from 2 weeks in advance up to 48h
- Booking page is valid from 14.09 to 21.12 2026

Lectures and instructors

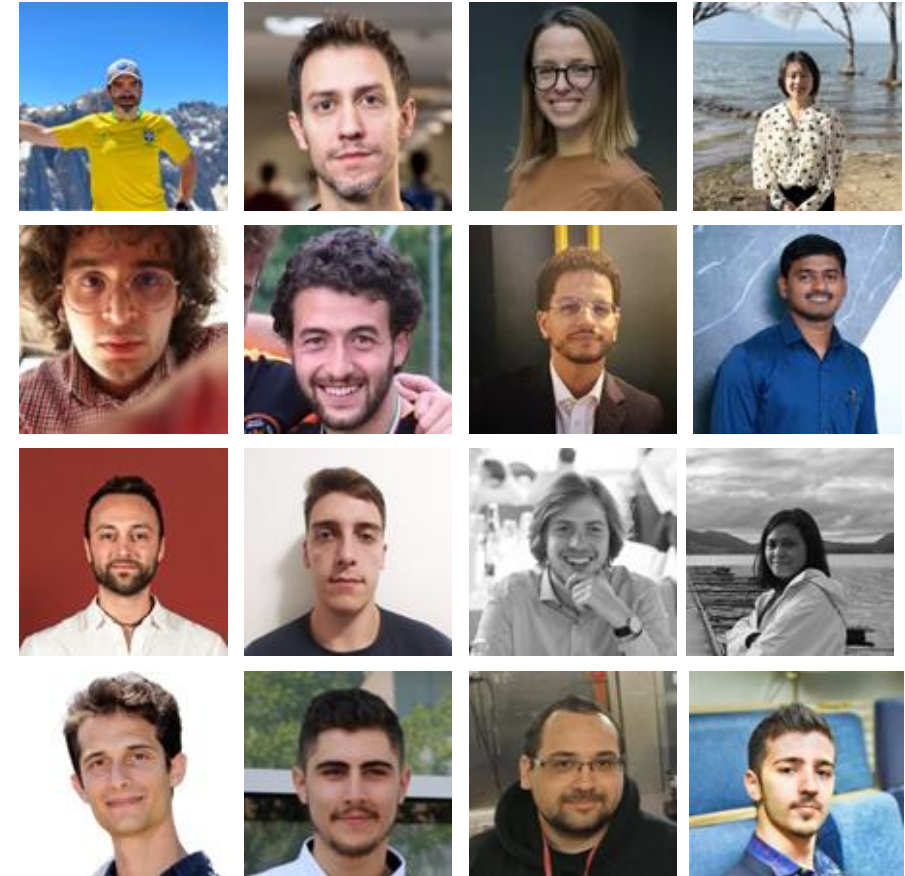
Associate Professor, lecturer in Algorithms and Parallel and GPU Computing

Education

- University of Perugia
- PhD, Sapienza University of Rome
- ETH Zurich, Switzerland

Previous roles

- Postdoc, CNR Institute for Applied Computing (IAC)
- Research Intern, NVIDIA, US
- Research Engineer, Didividi, Cambridge, UK
- Researcher, Free University of Bozen-Bolzano



HiCREST Trento Laboratory

Research areas

HPC and Parallel algorithms (Prof. Vella) and reliability (Prof. Rech) for parallel architectures.

- Parallel and distributed algorithms for emerging parallel systems
- Systems for Machine Learning
- GPU and Multi-GPU Computing
- Quantum Simulation and Quantum Circuits Reliability
- Reliable aerospace applications

Projects

- ARCHYTAS — ARCHitectures based on unconventional accelerators for dependable/energy-efficient AI Systems
- NET4EXA - A new generation of fast interconnect for HPC and AI systems



Lectures and instructors

Dr. Brahmaiah Gandham

Post-Doc at DISI – UNITN

Education

- Ph.D. in Computer Science and Engineering (HPC), Mahindra University, Hyderabad, India.
- Master of Technology in Computer Science and Engineering, JNTUK, India.
- Bachelor of Technology in Computer Science and Engineering, JNTUH, India.

Research interest

- Synchronization mechanisms.
- High-Performance Computing.
- Concurrent Data Structures.
- Shared Memory Programming

Lectures and instructors



UNITRENTO

Lorenzo Pichetti

PhD candidate at DISI – UNITN

Education

- Internship at the Italian National Research Council (CNR).
- Master of Science in Mathematics and Cryptography, Rome, Italy.
- Bachelor of Science in Computer Mathematics, Rome, Italy.

Research interest

- Sparse Linear Algebra applications;
- Parallel distributed-memory and multi-GPU programming;
- Communication-bound problems and algorithms.

Lectures and instructors



UNITRENTO

Jacopo Raffi

PhD at Candidate DISI – UNITN

Education

- Master of Science in Computer Science (Artificial Intelligence), Pisa, Italy.
- Bachelor of Science in Computer Science, Pisa, Italy.

Research interest

- AI accelerators.
- Deep Learning.
- High-Performance Computing.



Administrative information

When and Where

Course information

The last lecture **10 Dec 2026**

Two recovery dates: **14-17 Dec 2026**

No Lectures

1 October 2026

2 – 11 November Sessions on Special Lab Sessions

Week organization and rooms

Monday 8:30 – 10:30 Room A108

Thursday 10:30 – 12:30 Room A110

Course Background

Mandatory

Fundamentals of computer system architectures

Fundamentals of algorithms and data-structures (mainly arrays and matrices)

C / C++ programming language (pointers) and fundamentals on compilers

Operating systems

Other desired skills are:

Compilers (e.g., GCC)

Linux utilities (e.g., SSH, text editors) and build essential tools (make)

Knowledge of pthreads and mutex

Course Organization



UNITRENTO

Lectures

- **Frontal Lectures** are organized as follows:
 - Computer Architectures
 - Shared-memory parallel models
 - Distributed-memory models
 - Principles of Programming
 - Algorithms and Selected Topics
- **Labs and hands-on** are typically used to introduce the homework

Seminar and Guest Lectures

- TBD

Project activity will be presented during one lab session

Course material

The **lectures** (theory) are from

- An introduction to Parallel Programming. 2nd Edition 2022 (Peter Pacheco)
- Introduction to parallel computing 2nd edition (Grama, Karpis, Kumar, Gupta);
- Computer Architecture: a quantitative approach. 6th ed (Hennessy, Patterson);
- Parallel Programming in C with MPI and OpenMP 2003 (Michael J. Quinn)

Other useful resources:

- Programming Massively Parallel Processors: A Hands-on Approach 4th Edition (Kirk, Hwu)
- Parallel Computer Organization and Design (Dubois, Annavaram, and Stenstrom)

Slides and additional material (e.g., programming guides, howtos, papers, videos) will be available on the **Moodle platform** or listed at the end of the slide "**Extra reading material**"

I encourage you to use both the slides and the textbook to get the most out of your education

Course material

Technology and Tools

- Compilers GCC ([GCC website](#) · [GCC command-line documentation](#))
- Building tools: make or cmake ([Simple Makefile tutorial](#) · [GNU Make manual](#) · [CMake tutorial](#))
- Modules ([Environment Modules documentation](#) · [UniTN HPC Service Card](#))
- Secure shell (SSH) and linux editors (vim, emacs, nano)
- Resource Management Systems such as PBS ([PBS Professional User's Guide](#) · [UniTN HPC Service Card](#))
- Github ([Git basics](#) · [GitHub Getting Started](#))

Platform we will use for the homework and final project development

- UniTN cluster ([UniTN HPC Service Card](#))
- HPC Garage (equipped with [NVIDIA GPUs A100](#) and AMD [MI100](#))

Note Accounts will be created only for enrolled students!

Assessment and grades

Exams and evaluation:

- **Two options are available**
- **Both the options require to solve problems, deliver code, and presenting results**

Project can be related to

- Runtime techniques (scheduling, algorithms for managing concurrency)
- Fundamental routines and building block primitive (e.g., operations on arrays)
- Simple Algorithms (linear algebra and scientific computing, cryptography, deep learning, data science and analytics)

Each student will work **independently** and should produce a short document

- Methodology
- Quality
- Originality (depending on the state-of-the-art and other solutions)

Assessment and grades

Option 1

- **Project plus an oral presentation.**
- The project list will be available **the last week of October.**
- Each student will ask to get one among the available projects.

Report

Each student will work **independently** and should produce a short document (**report**)

- Methodology
- Quality
- Originality (depending on the state-of-the-art and other solutions)

Code and Reproducibility

- Students must provide link to git repository code and instructions to reproduce the results in the paper
- If the code will not compile or produce different results will not grade 100%
- We will check correspondence between code and the algorithm described in the report

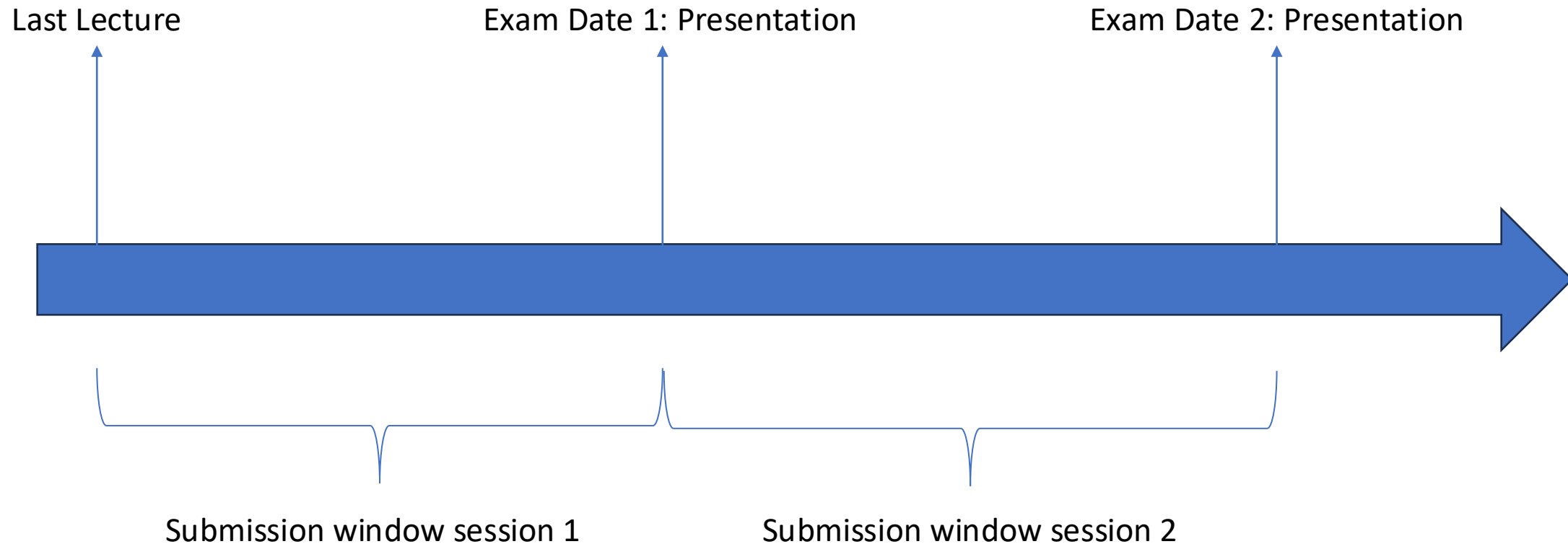
Assessment and grades



UNITRENTO

Submission and Presentation of the project

- Report and code must be submitted using the [registration form](#) the day before the exam
- Presentation: 10 min + Question on the code developed and the theory.



Assessment and grades

Option 2 – Deliverable

- Students will work on the same problem/task
- During the lab, one exercise will be related to the task
- Students will take a test with **closed-ended questions** at the end of the course (exam date)
- Submit the deliverable and the report in the the time window between **the two exam dates using the [registration form](#)**
- **To help students in the writing, optionally students can send a partial deliverable on shared-memory**

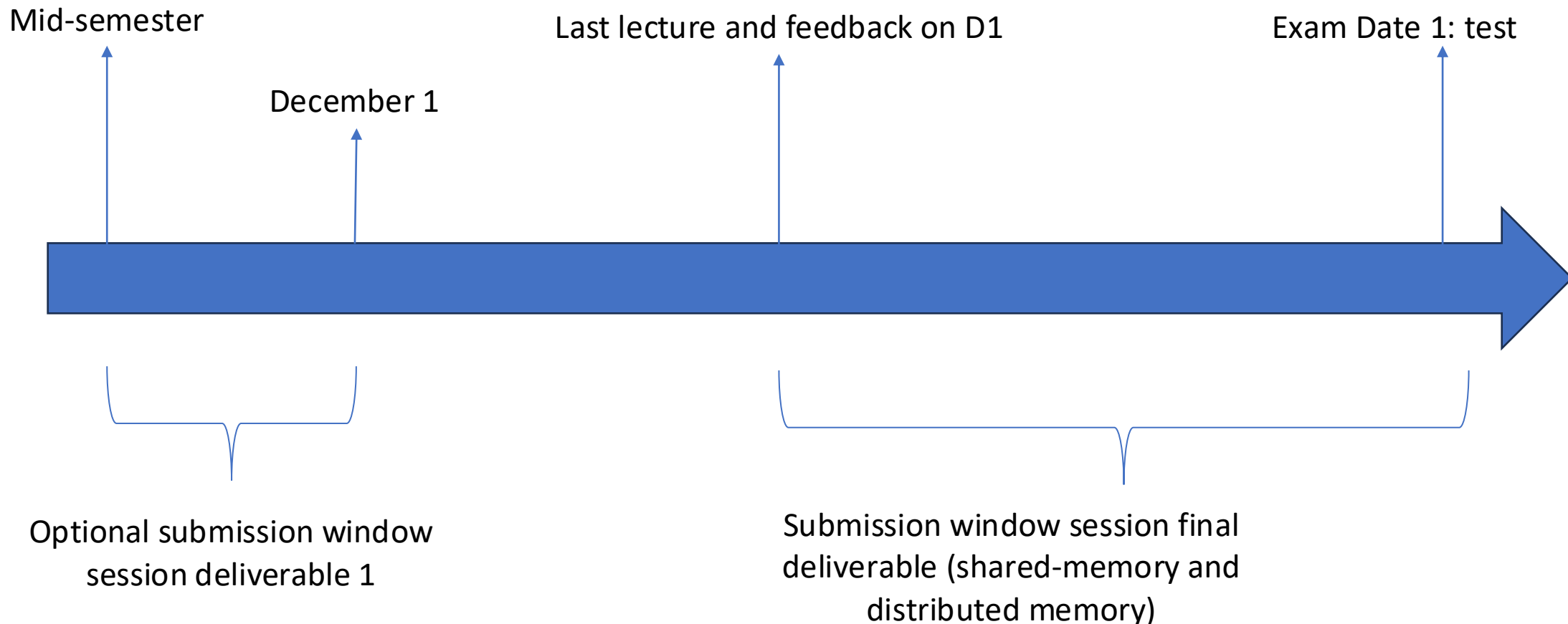
Each student will work **independently** and should produce a short document

- Methodology
- Quality
- Originality (depending on the state-of-the-art and other solutions)

Assessment and grades

Option 2

- Report and code must be submitted using the [registration form](#) the day before the exam
- Presentation: 10 min + Question on the code developed and the theory.



Grading table



UNITRENTO

Option 1:

- Report: 40%
- Presentation: 40%
- Code, Reproducibility and Correspondence: 20%

Option 2:

- Test: 30%
- Report: 60%
- Code, Reproducibility and Correspondence: 10%

Report

- Presenting parallel algorithms and reporting performance in an effective way is challenging
- Learning outcomes of the course include the ability to communicate and disseminate results.
- Both the exam options require students to write a 4-page report
- References are not included
- Upload the report in the Registration Form [Link](#)

Supporting activities

- 1 lab will focus on understanding scientific report
- TAs will provide feedback (option 2).
- Deliverable 1 will not be considered in the final grade
- We will provide a template for the self-assessment

Assessment and grades

FAQ

Q1: *What happens if I plan to give the exam in the other sessions (no winter)*

A1: "In case students will not opt for the deliverable or they would like to take the exam **AFTER** the semester when the course is offered, the exam will consist of an **oral** and a **project (option 1)**."

Q2: What happens if I do not pass the test in the option 2 exam.

A2: "You will not lose the deliverable grade, but you have to take the oral in the next coming sessions".

Q3: *I am a PhD student, and I would like to include this course in my study plan.*

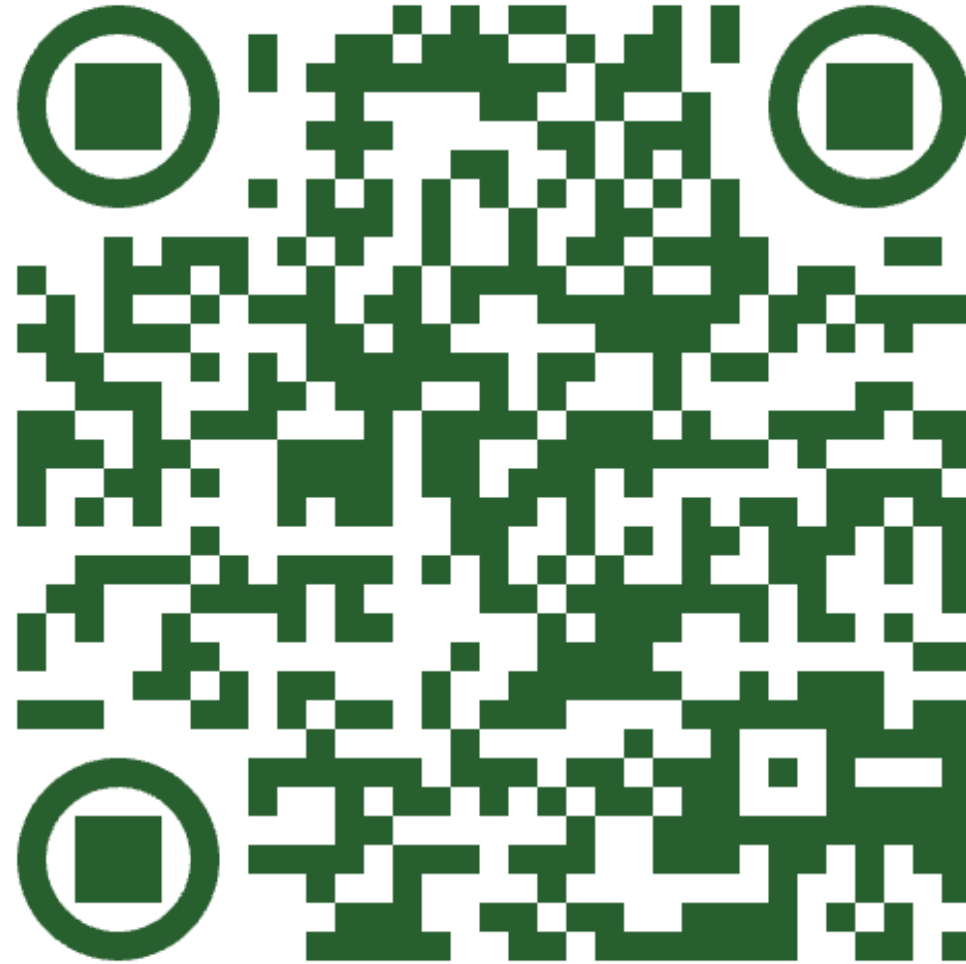
A3: "You are welcome, but you need to attend at least 70% of the lectures (~15 lectures). The exam consists of a scientific paper, or a specific topic related to the course.

Alternatively on a project activity".

Course Registration



UNITRENTO



<https://forms.gle/EGiph1sNroMbRnTm8>

Overview of the lecture

What is parallel computing

- Introduction and supercomputers
- Parallel Architectures
- From Parallel Architectures to Parallel Algorithms and Applications
- Programming perspective: parallel programming models
- Performance Considerations

Topics covered in the course

Course objectives

The objective of this course is to acquire the **methodologies**, the **best practice** and the **technologies** for the **analysis**, the **design**, the **development** of **parallel** algorithms able to exploit **parallel architectures**

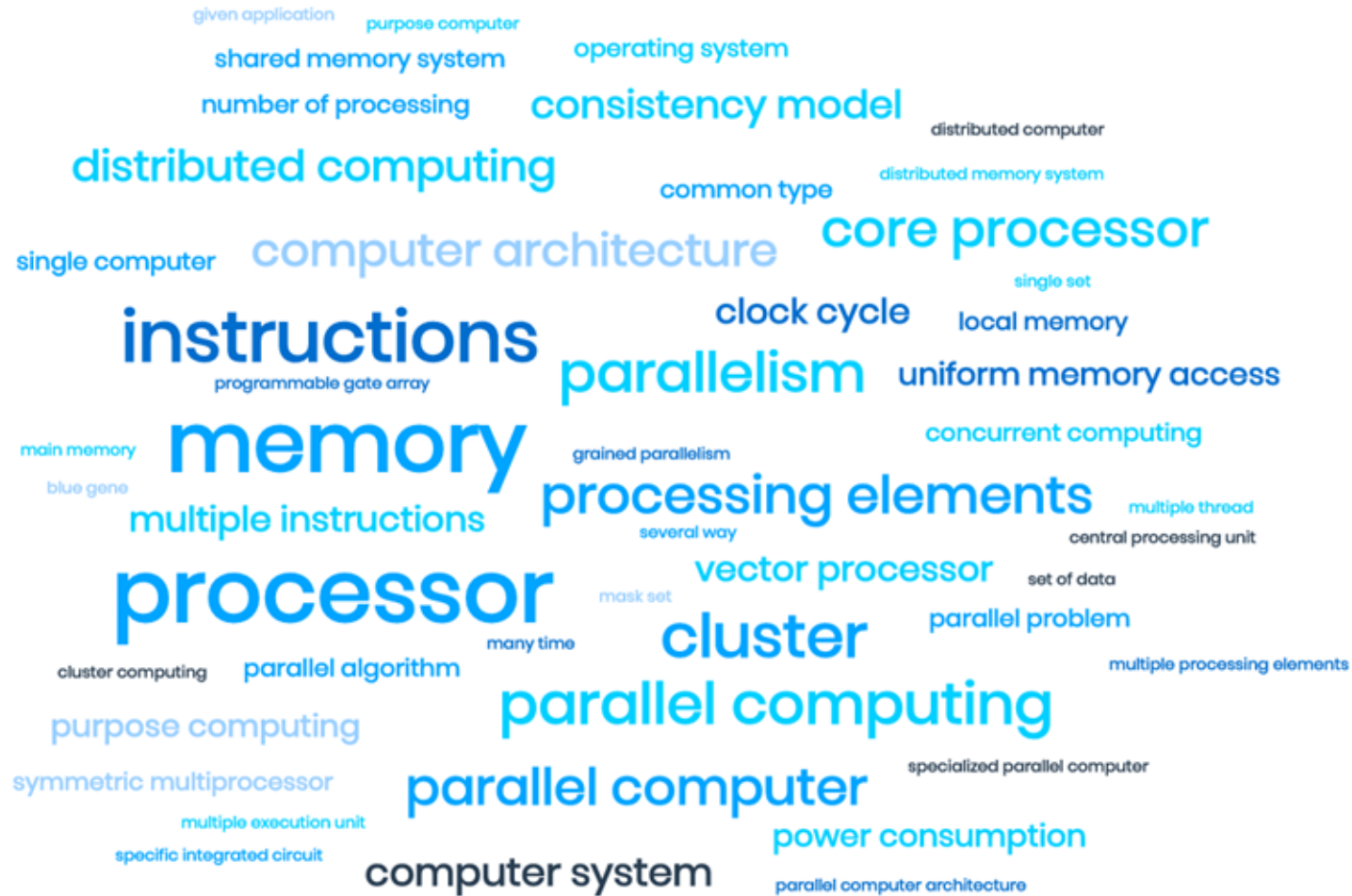
Introduction to Parallel Computing

Overview of the course

What is parallel computing?



UNITRENTO



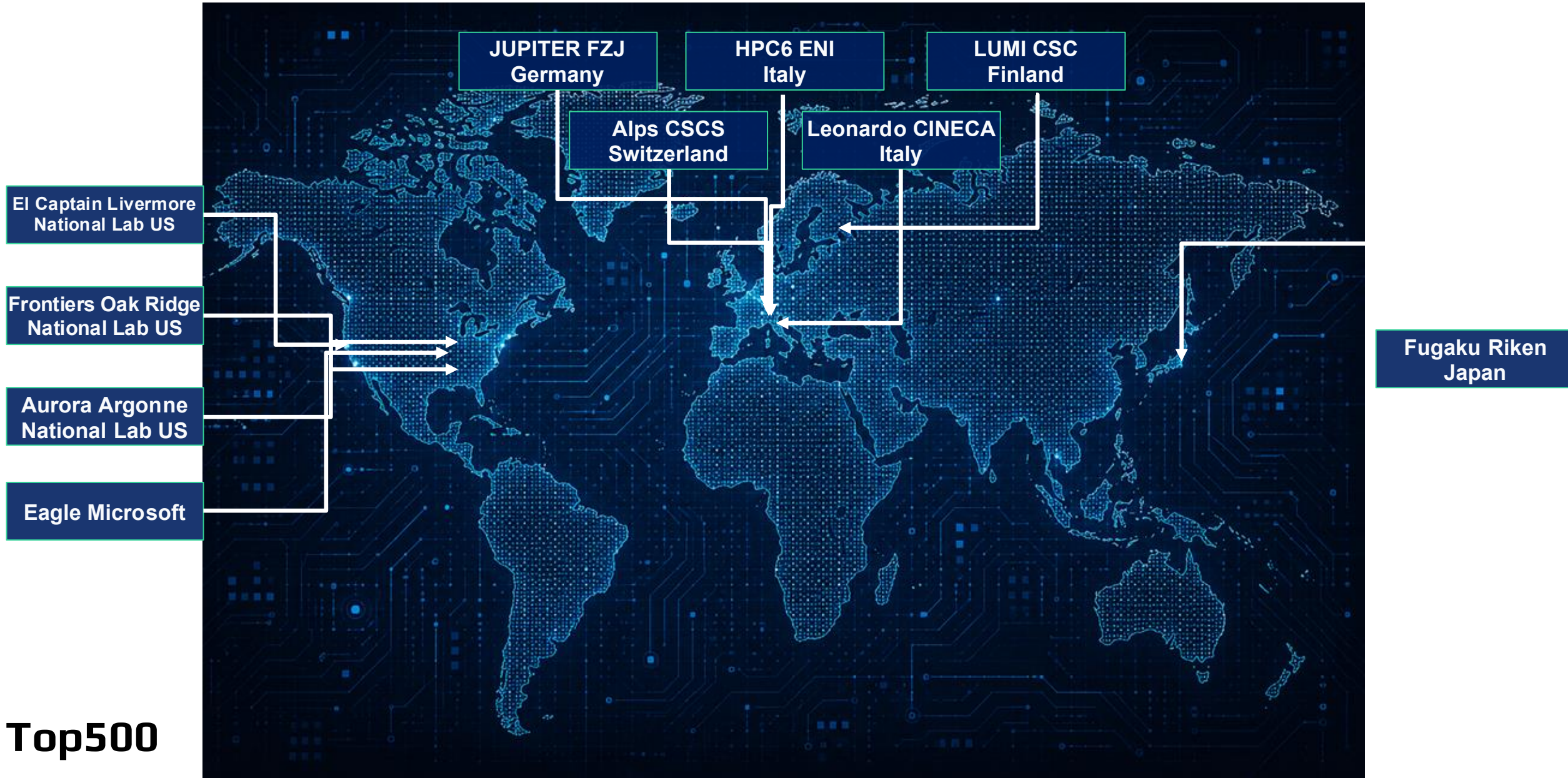
Source: [MonkeyLearn](#)

Text: [Wikipedia: Parallel Computing](#)

Supercomputers



UNITRENTO



The FLOPs scale



UNITRENTO

FLOPS = floating-point operations per second

10^{12}

teraFLOPS

10^{15}

petaFLOPS

10^{18}

exaFLOPS

10^{21}

zettaFLOPS?

Exascale Machine

It is estimated that the brain exceeds 10^{16} synaptic operations per second

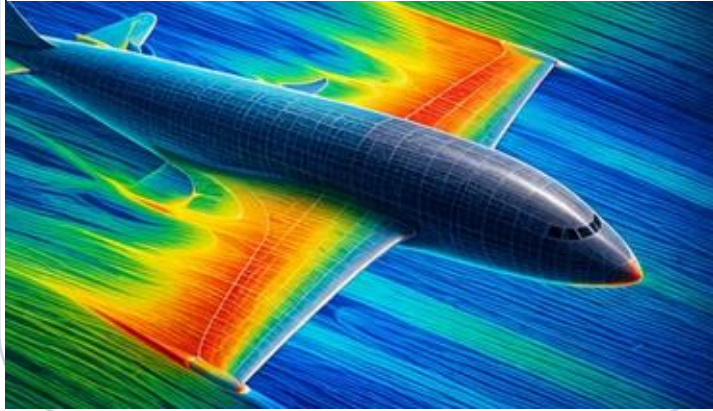
Each 1000× jump changes which problems become tractable.

What problems they solve



UNITRENTO

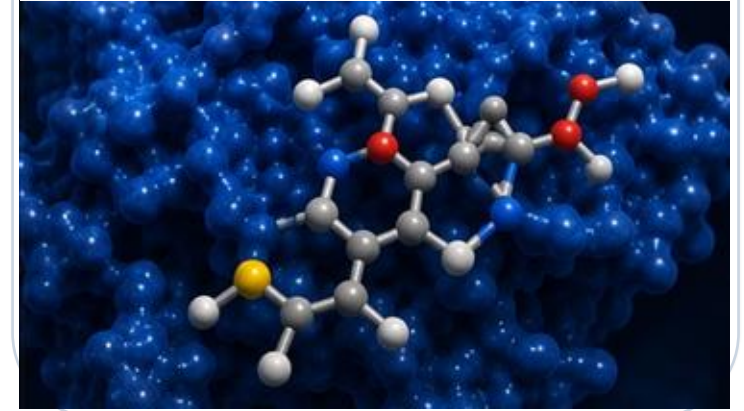
PHYSICAL SIMULATIONS



CLIMATE AND ENVIRONMENT



DRUG DISCOVERY



ARTIFICIAL INTELLIGENCE (AI)



DATA ANALYTICS AND BIG DATA



FORECASTING AND PREDICTION



Anatomy

a node = a powerful server

Node 1
CPU + GPU

Node 2
CPU + GPU

Node 3
CPU + GPU

Node 4
CPU + GPU

Node 5
CPU + GPU

Node 6
CPU + GPU

Node 7
CPU + GPU

Node 8
CPU + GPU

Node 9
CPU + GPU

Node 10
CPU + GPU

Node 11
CPU + GPU

Node 12
CPU + GPU

Every node

Processors (CPU), Accelerators (GPUs), Memory Hierarchy and Interconnects (Network Interface Card)

Key Idea

Each node solve a sub-problem

Parallelism

Multiple nodes work together to solve the entire large problem

Inside the node

CPU = Central
Processing Unit
control + logic

CPU

- Few, highly versatile cores
- Excellent for running many different applications via the operating system
- CPU coordinates the workload

Memory
RAM + HBM
DMA

Network
NICs

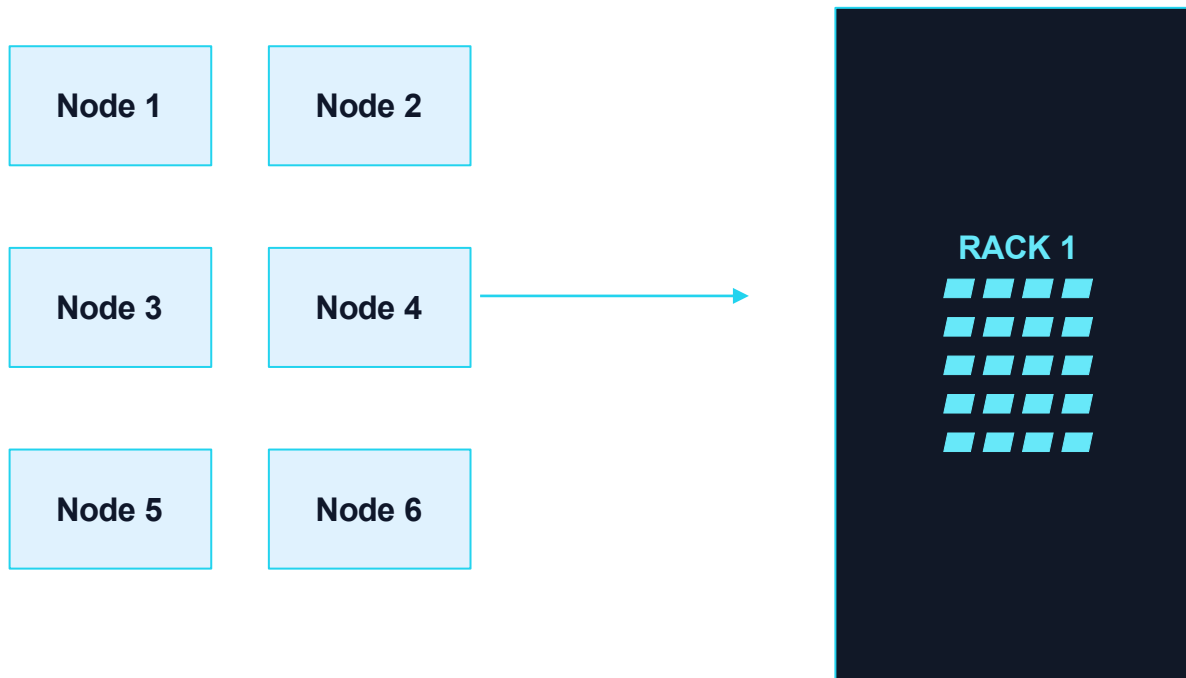
Graphics Processing Unit or
other accelerators
Massive parallel acceleration

GPU

- Thousands of simple cores
- Excellent for structured data like images
- Accelerates AI and simulations

**Supercomputers typically have both
Shared-memory programming models**

From the node to rack



Rack integrates

multiple nodes, power and sockets, a cooling system, switches and network components.

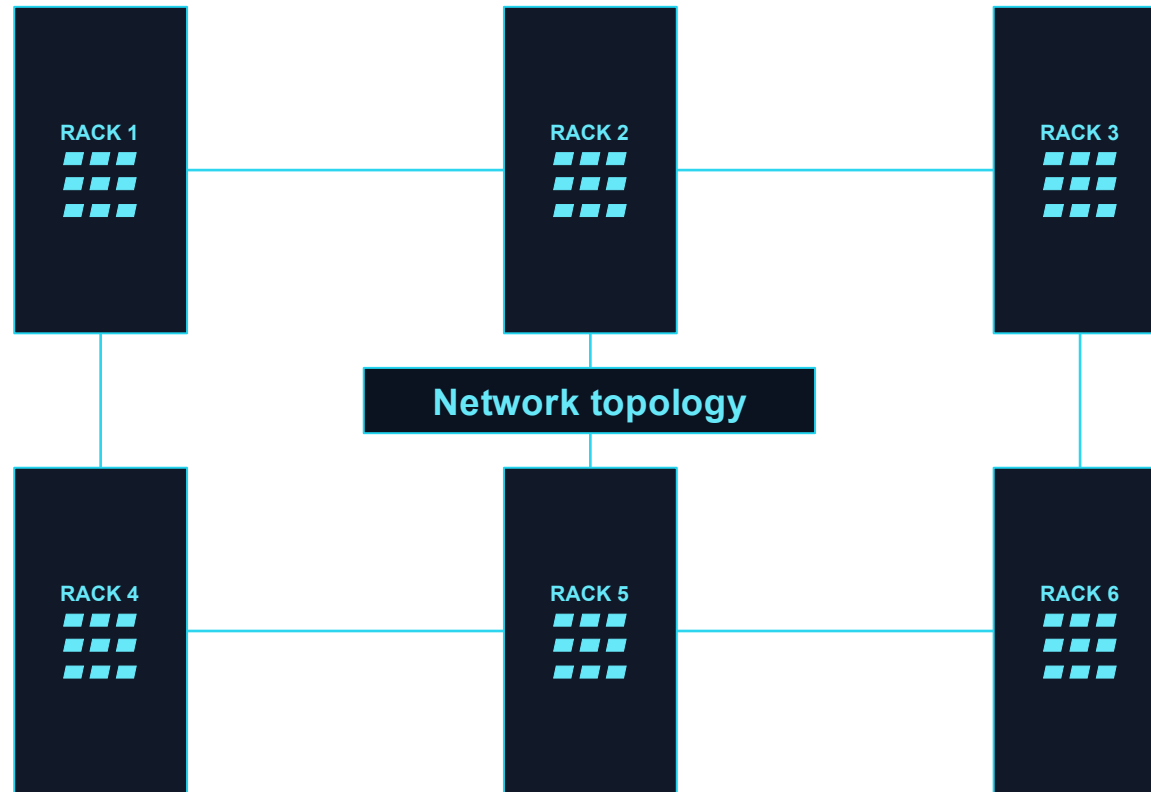
Why is it needed

It makes the system manageable: space, energy, cabling, and maintenance.

From racks to topology



UNITRENTO



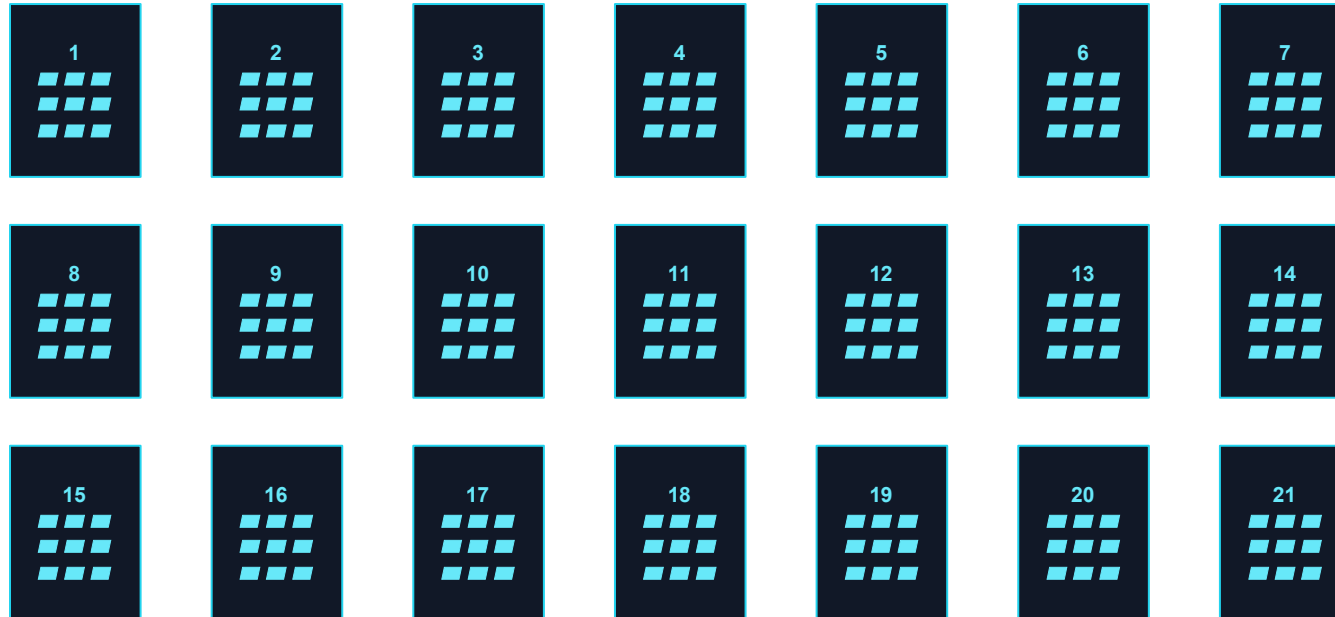
Goal

Low latency, high-bandwidth, multi-path to avoid congestion and bottlenecks

Examples

Different shapes
Mesh, torus, fat-tree, dragonfly: different topologies for different needs and applications

Fully stack system



Supercomputer looks like and acts as a single machine

Infrastructure
Energy and cooling.
Resources management

Parallel File systems

Nodes, racks, and network storage work together
Message Passing Programming Models to design distributed-memory applications

Questions the course wants to address

What do we need to know about the system to design efficient algorithms?

How to program a single parallel system?

What are the metrics and goals we are interested in?

How to program multiple parallel machines?

How can we coordinate all these activities on such a complex systems?

Topic covered in the course

Parallel architectures

Multicore, accelerators, interconnects

Parallel programming models

Shared memory, distributed memory, MPI

Parallel algorithms

Decomposition, patterns, scientific computing, AI/data science

Performance

Speedup, efficiency, scalability, communication, portability/productivity

What this course is about

We will not design new parallel architecture but adopt a

Hardware-Software co-design approach

- Traditionally parallelism has been addressed from an architecture perspective
- Today performance depends on several components

Algorithm design, data decomposition

Libraries and Primitives

Programming models and implementations

Run-time and Compilers

Architecture and Network Interconnection

What this course is about



UNITRENTO

In this course

80%

Algorithm design, data
decomposition

Libraries and Primitives

Programming models and
implementations

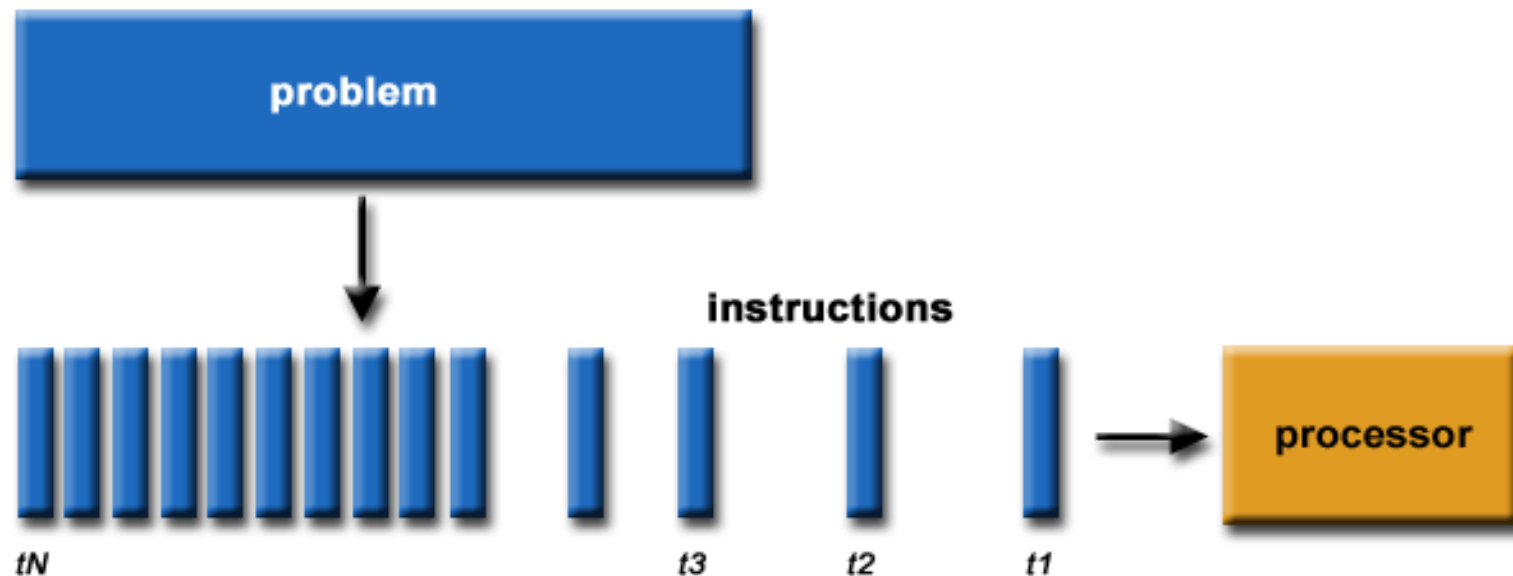
20%

Run-time and Compilers

Architecture and Network
Interconnection

Parallel Architectures and Parallel Computing

Introduction and motivation



Serial Computing

1. a program is broken into a sequence of *logic* instructions
2. Instructions are executed sequentially one-by-one
3. Executed on a single processor
4. Only one instruction may execute at any moment in time

Introduction and motivation

Suppose an e-commerce company and its logistic.

*Boxes of equal size that contain goods and products are organized into big storage. An employee **P** should take the box from a storage area and move them to a load area to be shipped through a track service.*

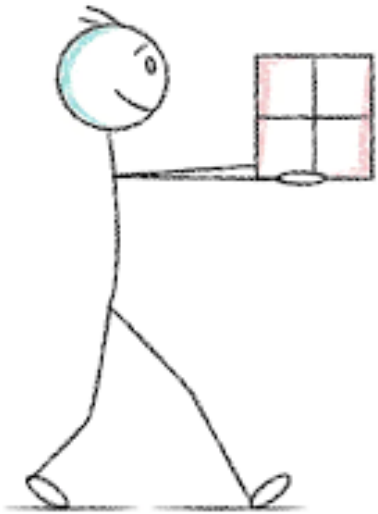
Class discussion

Assuming we have one employee, split this task in a simple sequence of actions.

Introduction and motivation

Task decomposition as sequential steps

1. P takes a box from the storage;
2. P goes to the area for the loading;
3. P drops the box
4. P comes back to the storage



Introduction and motivation

Suppose an e-commerce company and its logistics.

*Boxes of equal size that contain goods and products are organized into big storage. An employee **P** should take the box from a storage area and move them to a load area to be shipped through a track service.*

Class discussion

What considerations we must do in order to analyse the cost of this activity?

Introduction and motivation

Task decomposition as sequential steps

1. P takes a box from the storage (time and capacity);
2. P goes to the area for the loading (time and distance);
3. P drops the box (time)
4. P comes back to the storage (time and distance)

Introduction and motivation

Task decomposition as sequential steps

1. P takes a box from the storage (2 sec per kg);
2. P goes to the area for the loading (10m per second);
3. P drops the box (2 sec)
4. P comes back to the storage (10 meter per second)

Class discussion

Compute the total time assuming assuming that the distance from the storage to the load area is 100 m?

Introduction and motivation

Task decomposition as sequential steps

1. P takes a box from the storage (2s);
2. P goes to the area for the loading (10s);
3. P drops the box (2s)
4. P comes back to the storage (10s)

Class discussion

Compute the total time assuming that the distance from the storage to the load area is 100 m?

24 seconds for each 1 Kg box.

If your truck contains 100 boxes, P needs 2400 seconds!

Introduction and motivation

Fact: "Parallel computing allows us to reduce time-to-solution, increase throughput, or solve larger problems by exploiting multiple computing resources"

Performance consideration

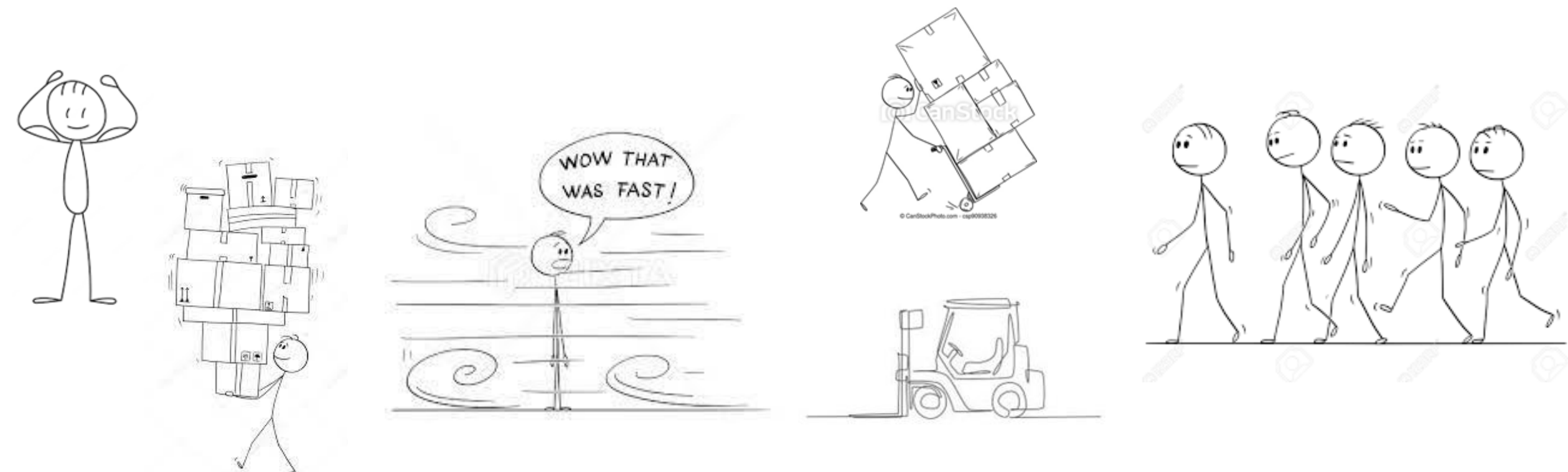
P = Take boxes from the storage; go to the area for the loading; Load the track; come back to the storage.

How can we improve the execution time of P per box?

Introduction and motivation

Possible solutions

1. Hire a faster and stronger employee
2. Buy and use a carry or a forklift
3. Hire more employees



Introduction and motivation

Possible solutions

1. Hire a faster and stronger employee
2. Buy and use a cart or a forklift
3. Hire more employees

On **computer systems**, such solutions correspond to

1. Hire a faster and stronger employee (**increase the clock of the processors to reduce the latency and increase the throughput**)
2. Buy and use a forklift (**vectorization, wider data paths, batching, or higher memory bandwidth**)
3. Hire more employees (**multiple processing elements to increase the throughput**)

Introduction and motivation

Other solutions

Change the entire system organization.

For example,

1. Adding more independent helpers(functional units)
2. reduce the path from the storage to the loading area (reducing the path length)
3. Add smaller temporary storage in the path (cache)

Performance of individual functional units depends on transistor technology.



Introduction and motivation

Performance of individual functional units depends on transistor technology.

Moore's Law – The number of transistors on integrated circuit chips (1971-2018)
 Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are

Moore's Law Facts: Transistor counts double every two years!

Smaller transistors $\Rightarrow$ can put more on chip $\Rightarrow$ computational power grows exponentially $\Rightarrow$ your sequential program automatically gets faster.

Also applies to memory capacity!

Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
 The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.



Introduction and motivation

Transistor linear dimensions could be scaled down by 30% ($0.7\times$)

- their area decreases by approximately 50% ($0.5\times$)
- the voltage, V , decreases by 30% ($0.7\times$) to keep the electric field approximately constant
- capacitance, C , decreases by approximately 30% ($0.7\times$)

Other implications are:

- circuit delay decreases by approximately 30% ($0.7\times$), allowing operating frequency, f , to increase by about 40% ($1.4\times$)
- dynamic power per transistor decreases by approximately 50% ($0.5\times$)
- with approximately $2\times$ transistor density, power density remains approximately constant (ideal Dennard scaling)

References

Original paper 1974: Dennard, Robert H et al. "Design of ion-implanted MOSFET's with very small physical dimensions". IEEE Journal of Solid-State Circuits." <https://ieeexplore.ieee.org/document/1050511>

McMenamin, Adrian "The end of Dennard scaling".

<https://cartesianproduct.wordpress.com/2013/04/15/the-end-of-dennard-scaling/>

Dynamic power: $P \propto C \cdot V^2 \cdot f$

Under ideal Dennard scaling: $0.7 \times 0.7^2 \times 1.4 \approx 0.5\times$ per transistor.

Introduction and motivation



UNITRENTO

Moore's law says that the number of transistors doubles approximately every two years that in combination of the Dennard observations

Empirical observation

Performance per watt grows even faster, doubling about every 18 months (1.5 years).

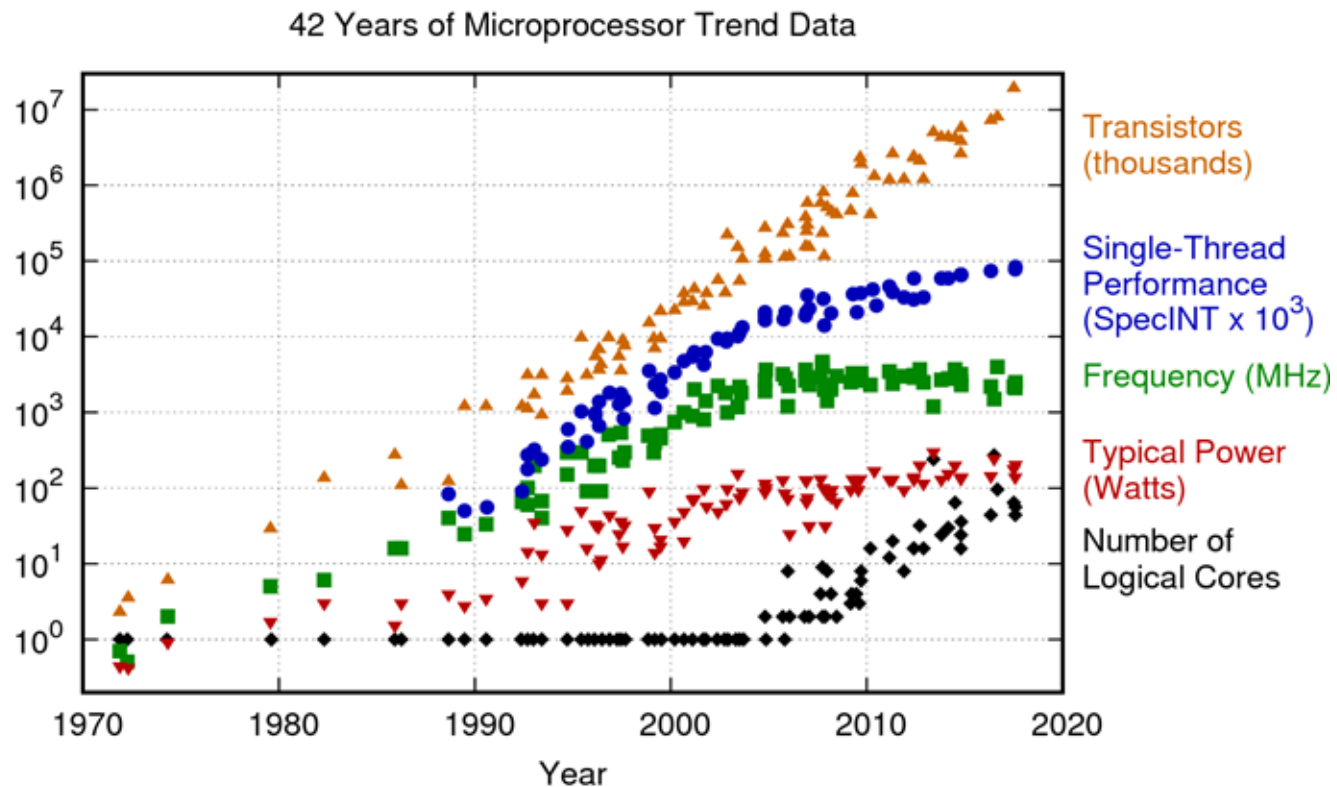
Is this about parallel computing?

Introduction and motivation

Why do not the we keep increasing clock frequency?

Transistors is getting **smaller** and faster:

- more efficient processors (energy consideration)

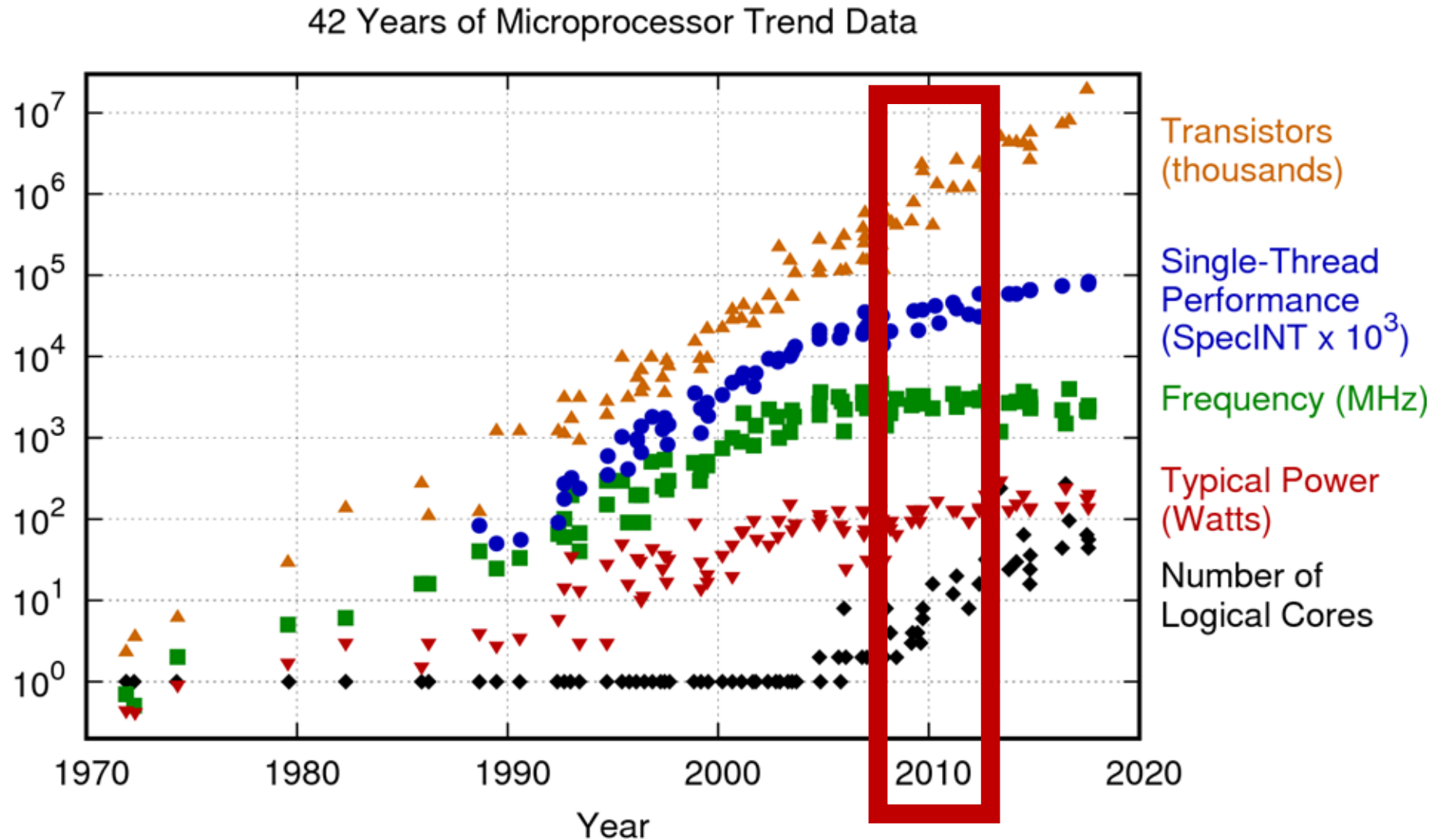


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

we can put more cores in the same die



UNITRENTO



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

Introduction and motivation

Main design constraint today is the cost and the power

- Single-Core CPUs:
- Complex Control Hardware
- Pro: Flexibility + Performance!
- Cons: Expensive (cost and power)

Multi/Many-Core

- Simpler Control Hardware
- Pros: replicating processing resources can provide better throughput and performance/W than spending the transistor budget only on increasing single-core complexity
- Cons: More restrictive/complex programming models and algorithm design

From More Cores to Parallel Execution

From more cores to parallel execution

Hardware perspective

- Power and cost constraints limit further single-core scaling.
- Multi/many-core architectures provide multiple processing elements (PEs).
- More PEs increase the potential amount of work that can be executed simultaneously.

Software perspective

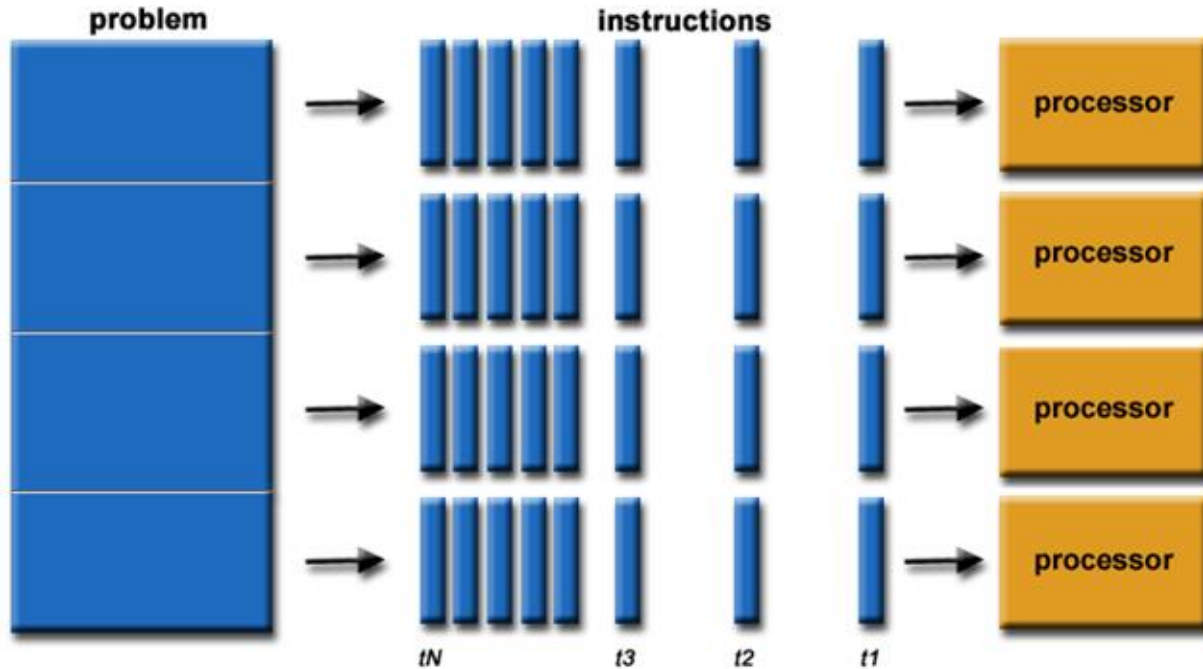
- Having multiple cores is not sufficient to obtain speedup.
- The application must expose multiple activities that we can process independently.
- These units of work expose concurrency

From more cores to parallel execution

Parallel hardware provides the resources

Concurrency in the algorithm provides the work to exploit them.

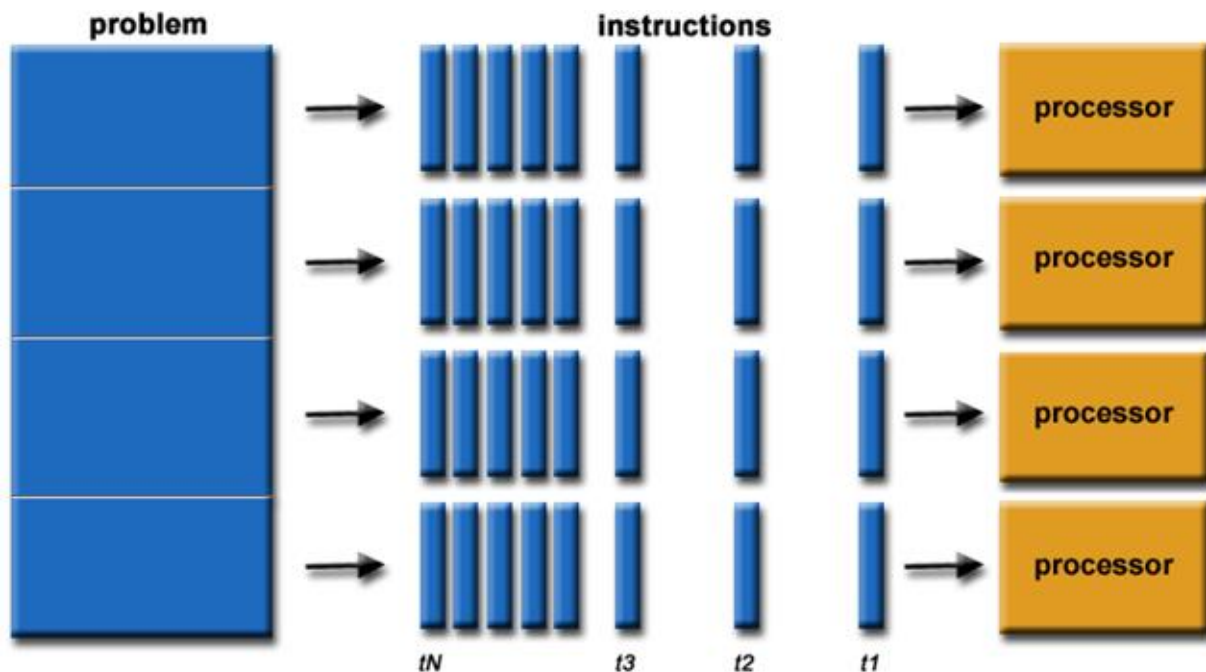
Introduction to Parallel Computing



Parallel Computing is the simultaneous use of multiple compute resources to solve a computational problem

- A program is broken into discrete parts that can be solved concurrently
- **Multiple instructions** are executed at any moment in time, depending on the number of processors
- An overall control/coordination mechanism is employed
- Each PE executes its own instruction stream while multiple PEs execute simultaneously.

Introduction to Parallel Computing



The program should be able to:

- decompose the pieces of work that can be solved simultaneously;
- Execute multiple instructions at any moment in time;

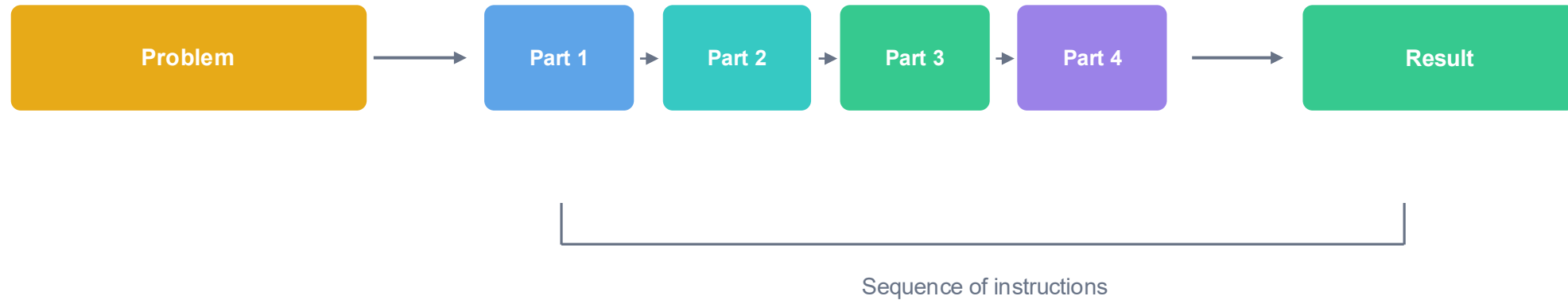
The goal is to solve the problem in less time by exploiting multiple compute resources.

Example of Parallel Algorithms

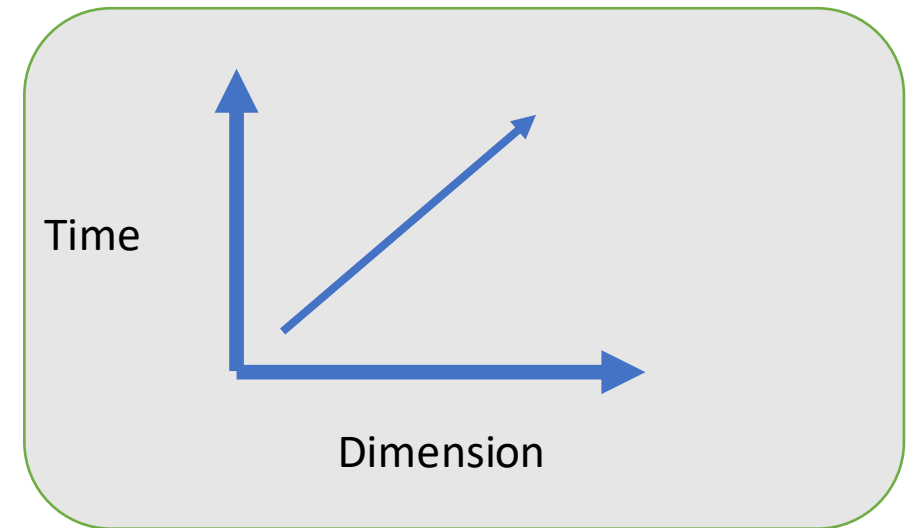
Class Activity

Find problems or algorithms that can be easily parallelized

From sequential to parallel algorithms



- Algorithms are a recipe for solving a problem through a sequence of simple operations.
- The larger and more complex the problem, the more time it takes to solve.

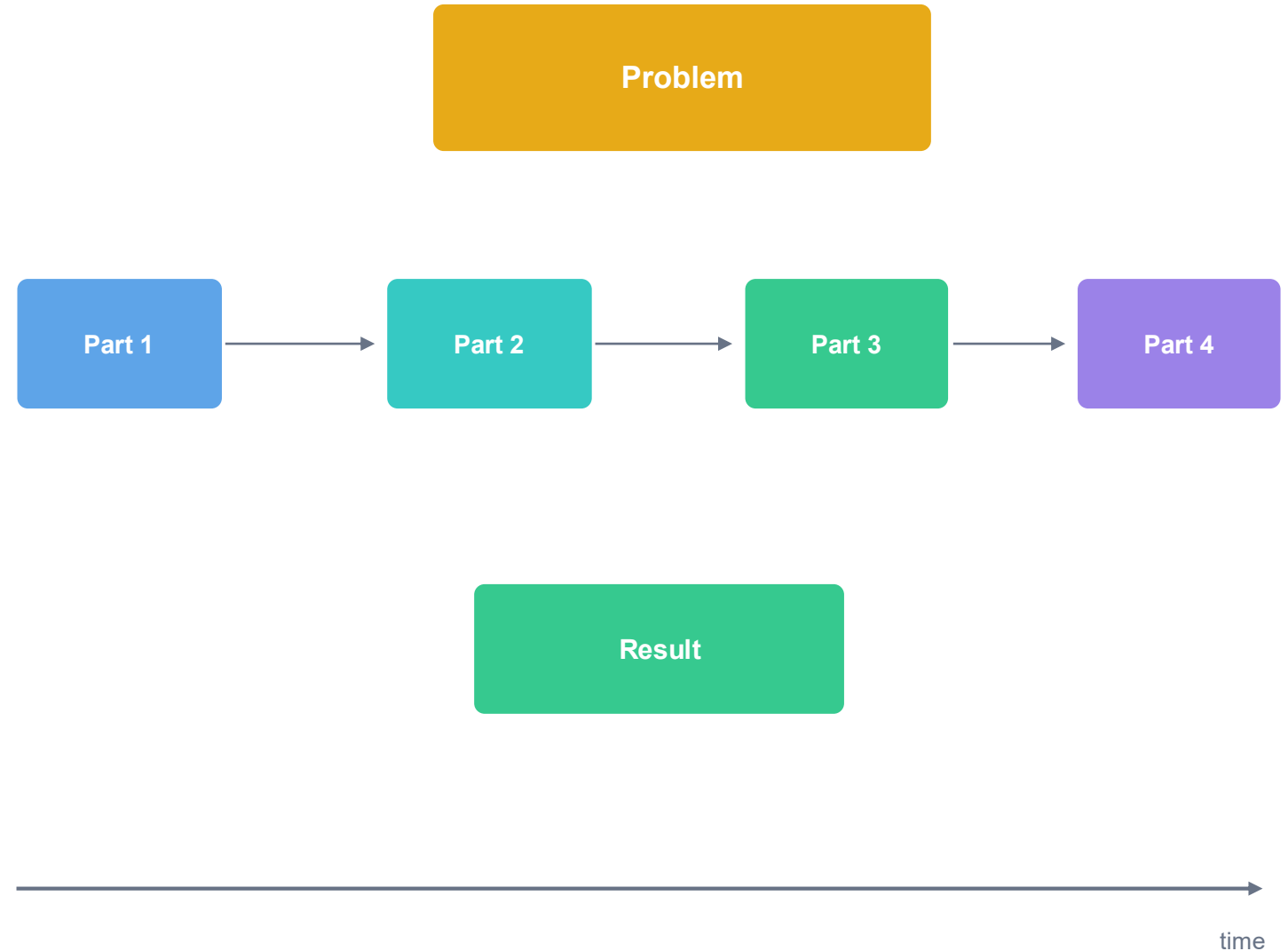


From sequential to parallel algorithms

Preparing pasta carbonara

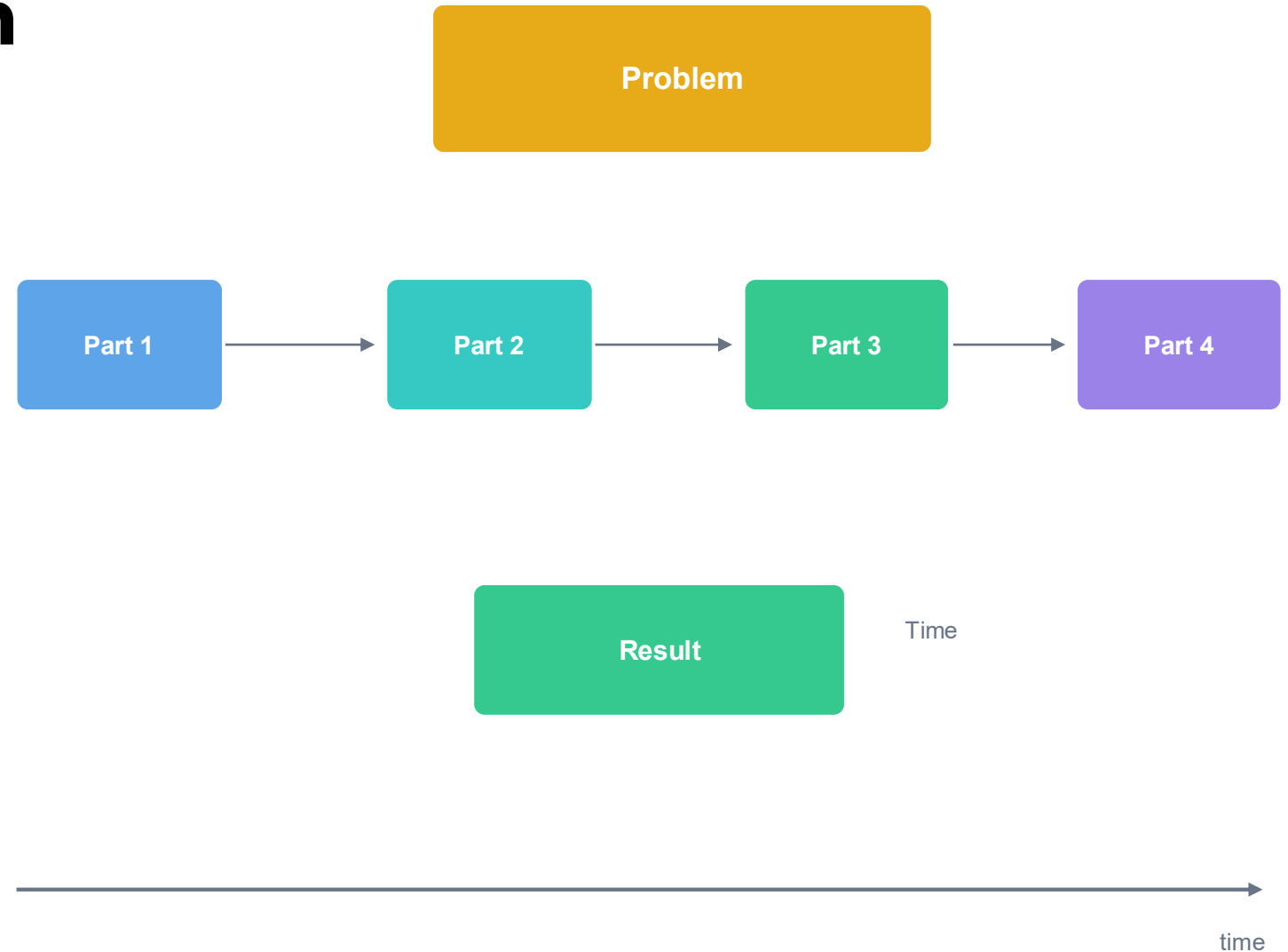
Algorithm / Recipe

1. Cook the pasta
2. Scramble the eggs
3. Pasteurize the eggs
4. Guanciale
5. Assemble



From sequential to parallel algorithms

Can we do better than this?



From sequential to parallel algorithms

Preparing pasta carbonara

Algorithm / Recipe

1. Cook the pasta
2. Scramble the eggs
3. Pasteurize the eggs
4. Guanciale
5. Assemble

Observation

Independent actions

- Cook the pasta and scramble the eggs

Dependent actions

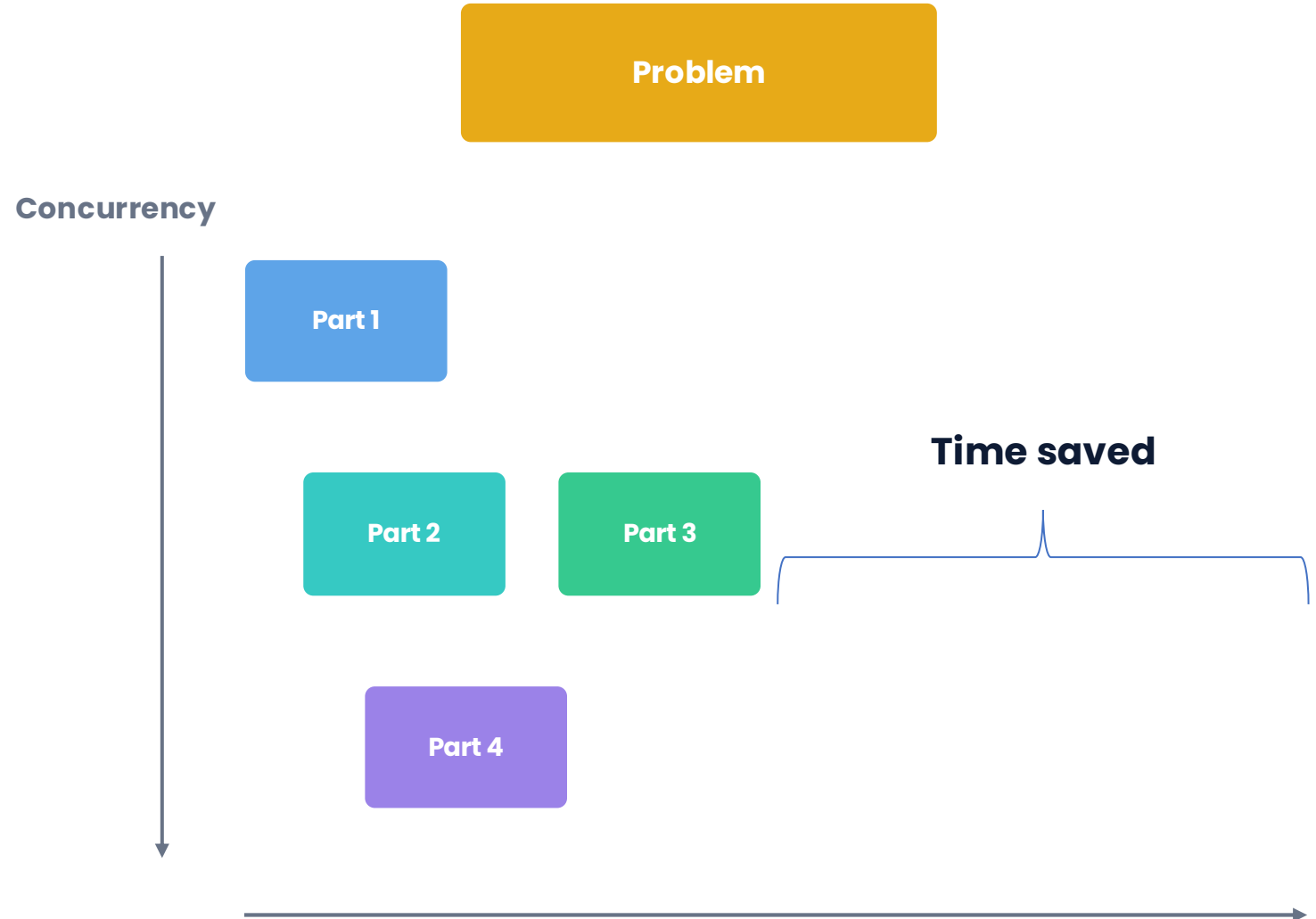
- Scramble the eggs and pasteurize them

From sequential to parallel algorithms

Preparing pasta carbonara

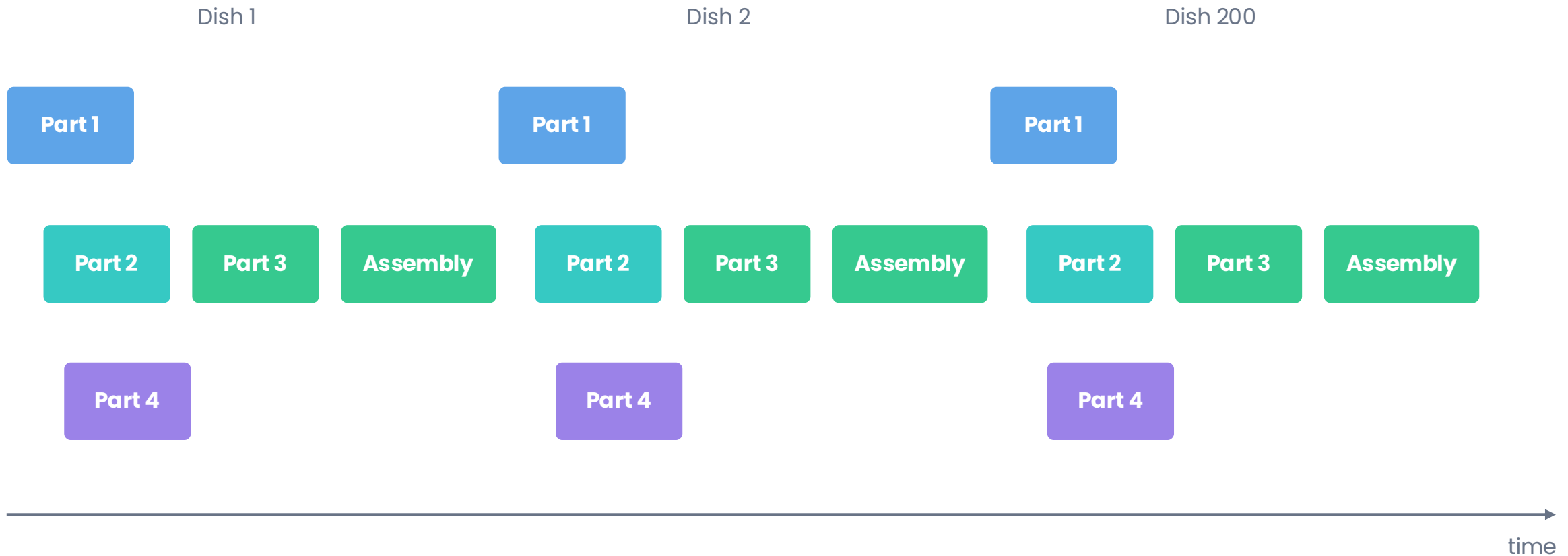
Algorithm / Recipe

1. Cook the pasta
2. Scramble the eggs
3. Pasteurize the eggs
4. Guanciale
5. Assemble



From sequential to parallel algorithms

Prepare 200 plates of pasta carbonara



From sequential to parallel algorithms



Task vs Data Parallelism



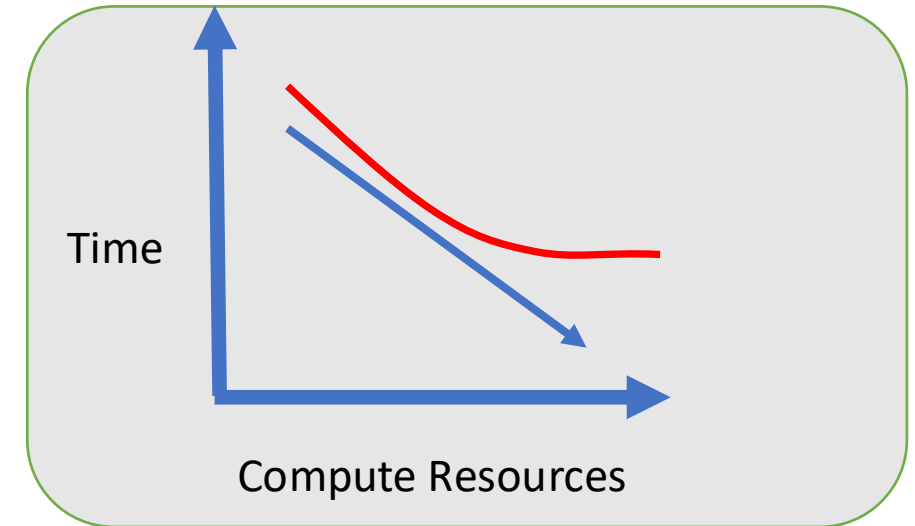
UNITRENTO



Task parallelism: different jobs at the same time

Data parallelism: the same operation on many data items

Scalability: how much performance improves as the number of nodes increases



Why could we not scale?

Parallel computing from software level

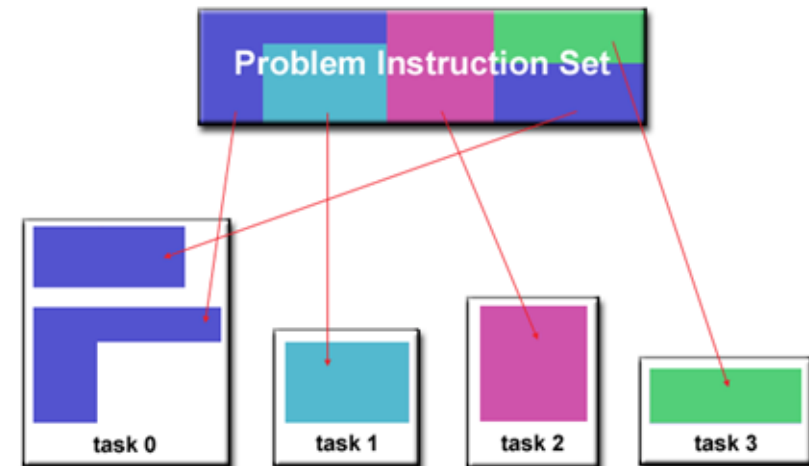
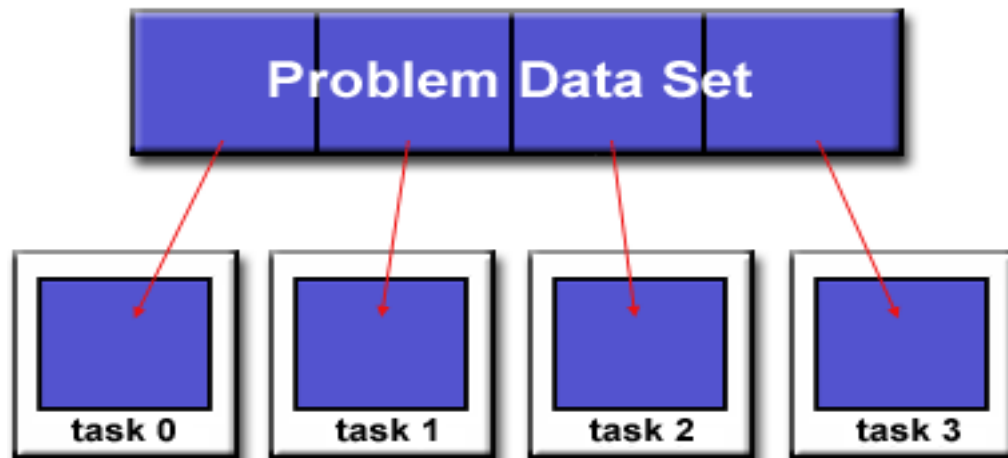
We can consider parallelism on the data or based on the type of activity:

- **Data decomposition:** the data is decomposed in small chunks.

"Each processing element (PE) performs a specific task on a different portion of the data"

- **Task decomposition:** the problem is decomposed according to the work that must be done.

"Each processing element (PE) performs a different task on a portion or on all the data"



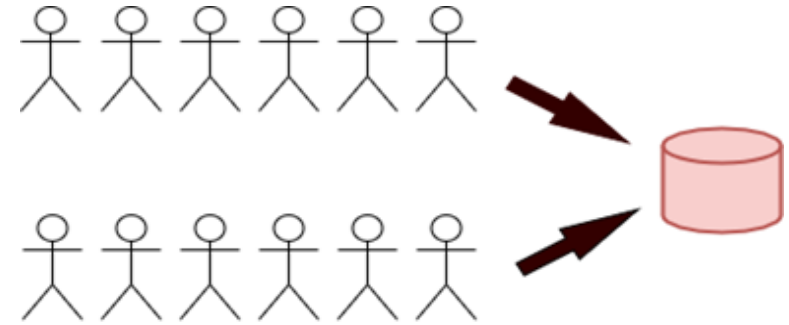
Concurrency and Parallelism



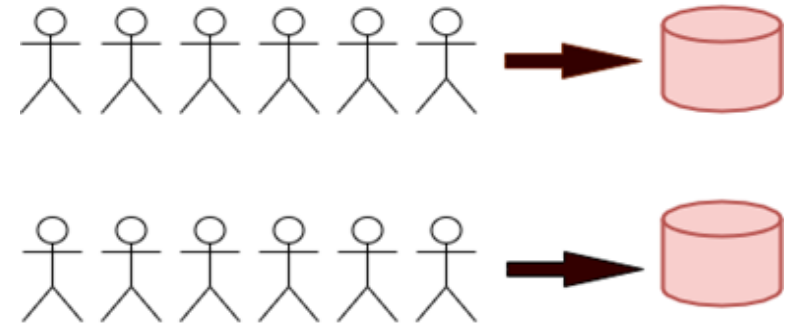
UNITRENTO

Informal definition

Concurrency (degree of) is multiple activities whose execution can overlap in time



Parallelism (degree of) is multiple activities actually executing simultaneously



Challenges

- Not all tasks are embarrassingly parallel
- We do not have infinite resources

Amdahl's law



UNITRENTO

95% parallel

5% sequential

- Even a few serial steps can slow down everything
- Communication and synchronization have a cost
- Good algorithm > simply adding hardware

Amdahl's Law

Execution-time model

1 processing
element



p processing
elements



s = serial fraction of the one-PE execution time

$1 - s$ = perfectly parallelizable fraction

p = number of processing elements

Ideal model: fixed problem size, perfect scaling of the parallel fraction, and no additional parallel overhead.

Parallel execution time

$$T_p = sT_1 + (1 - s)T_1 / p$$

Speedup

$$S(p) = T_1 / T_p = 1 / [s + (1 - s)/p]$$

Upper bound

$$\lim_{p \rightarrow \infty} S(p) = 1 / s$$

Key idea: the serial fraction sets a hard speedup limit.

Identify the serial and parallel fractions

Pseudocode

```
read_input(A, N)                // 10 s
– sequential

for i = 0 ... N-1
    B[i] = expensive_function(A[i]) // 85 s – parallelizable

write_result(B, N)              // 5 s – sequential
```

Measured on one PE

$$T_1 = 10 + 85 + 5 = 100 \text{ s}$$

From code to Amdahl

Sequential time

$$T_s = 10 + 5 = 15 \text{ s}$$

$$s = 15 / 100 = 0.15$$

Parallelizable time

$$T_{\text{par}} = 85 \text{ s}$$

$$1 - s = 0.85$$

Speedup model

$$S(p) = 1 / [0.15 + 0.85/p]$$

Serial fraction = 15%

Question to ask first: Which regions can actually execute concurrently?

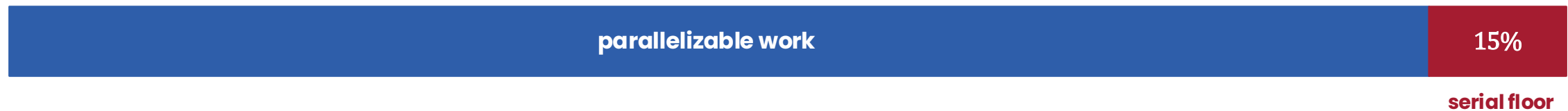
Amdahl's Law — worked example

Given $T_1 = 100\text{ s}$ $s = 0.15$ $1 - s = 0.85$

$$T_p = 15 + 85/p$$

1 PE	4 PEs	16 PEs	∞ PEs
100.0 s	36.25 s	20.31 s	15.0 s
1.00×	2.76×	4.92×	6.67×

Execution time cannot fall below the serial work



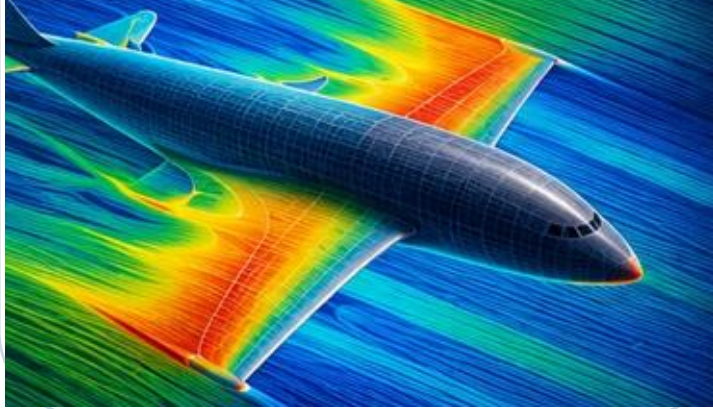
Adding processors gives diminishing returns: once the parallel part is very fast, the unchanged serial part dominates the total time.

Parallel Computing and Applications



UNITRENTO

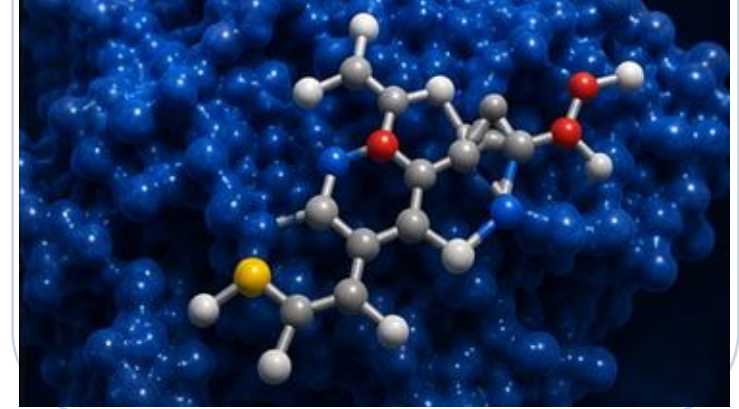
PHYSICAL SIMULATIONS



CLIMATE AND ENVIRONMENT



DRUG DISCOVERY



ARTIFICIAL INTELLIGENCE (AI)



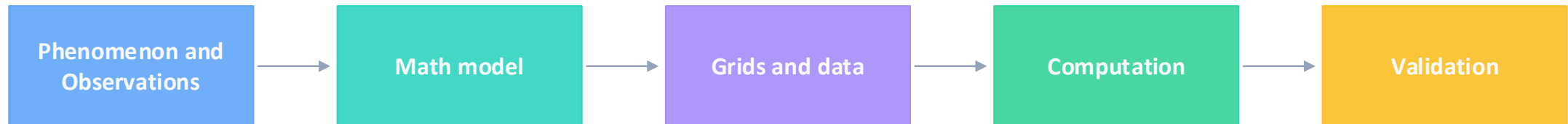
DATA ANALYTICS AND BIG DATA



FORECASTING AND PREDICTION



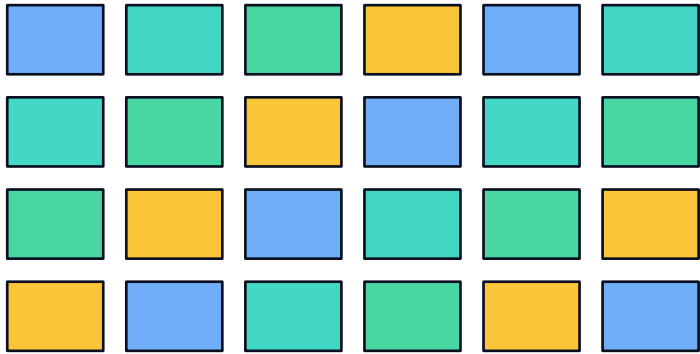
Parallel computing and applications



Climate



UNITRENTO



Atmosphere and oceans divided into millions of cells
Each cell evolves and exchanges information
Higher resolution = more detailed forecasts

Example: hurricanes, heatwaves, droughts, air quality.

From observation to parallel code

$$\rho(\partial \mathbf{u} / \partial t + (\mathbf{u} \cdot \nabla) \mathbf{u}) = -\nabla p + \mu \nabla^2 \mathbf{u} + \mathbf{f}$$

Meaning of symbols

$\mathbf{u} \rightarrow$ fluid velocity

$p \rightarrow$ pressure

$\mu \rightarrow$ viscosity

$\rho \rightarrow$ density The equations describe fluid motion.

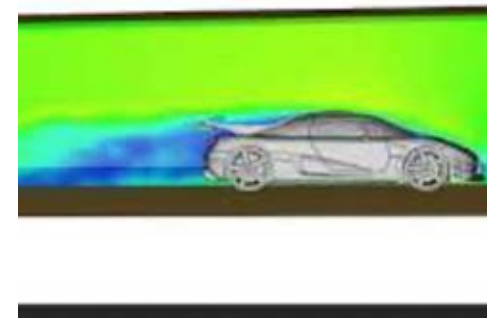
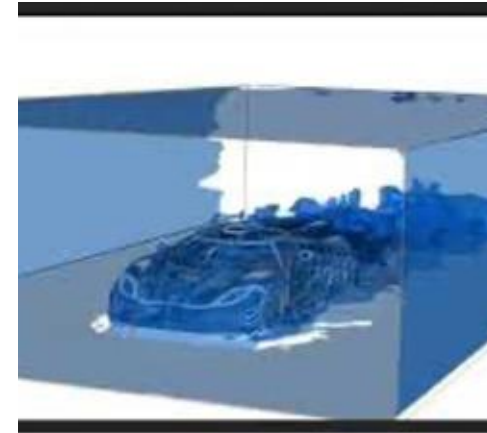
Computational Fluid Dynamics

- Simulation of the airflow around the car
- Reduction of aerodynamic drag
- Optimization of downforce
- Turbulence analysis
- Improvement of stability and efficiency

Parallel computing and applications

Computational Fluid Dynamics

- Simulation of the airflow around the car
- Reduction of aerodynamic drag
- Optimization of downforce
- Turbulence analysis
- Improvement of stability and efficiency



Abstraction and Parallelism

From parallel architectures to parallel programming models



UNITRENTO

Modern parallel architectures

- Multi Processors/multi-core architectures with bus interconnections (e.g., Intel, AMD, ARM, and RISC-V based processors)
- Multi Processors/multi-core systems and interconnects

Lumi supercomputer

<https://docs.lumi-supercomputer.eu/hardware/lumig/>

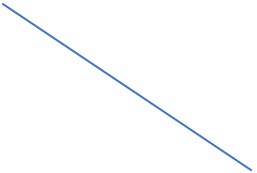
From parallel architectures to parallel programming models



UNITRENTO

Modern parallel architectures

- Shared-memory vs Distributed Memory



<https://docs.lumi-supercomputer.eu/hardware/>

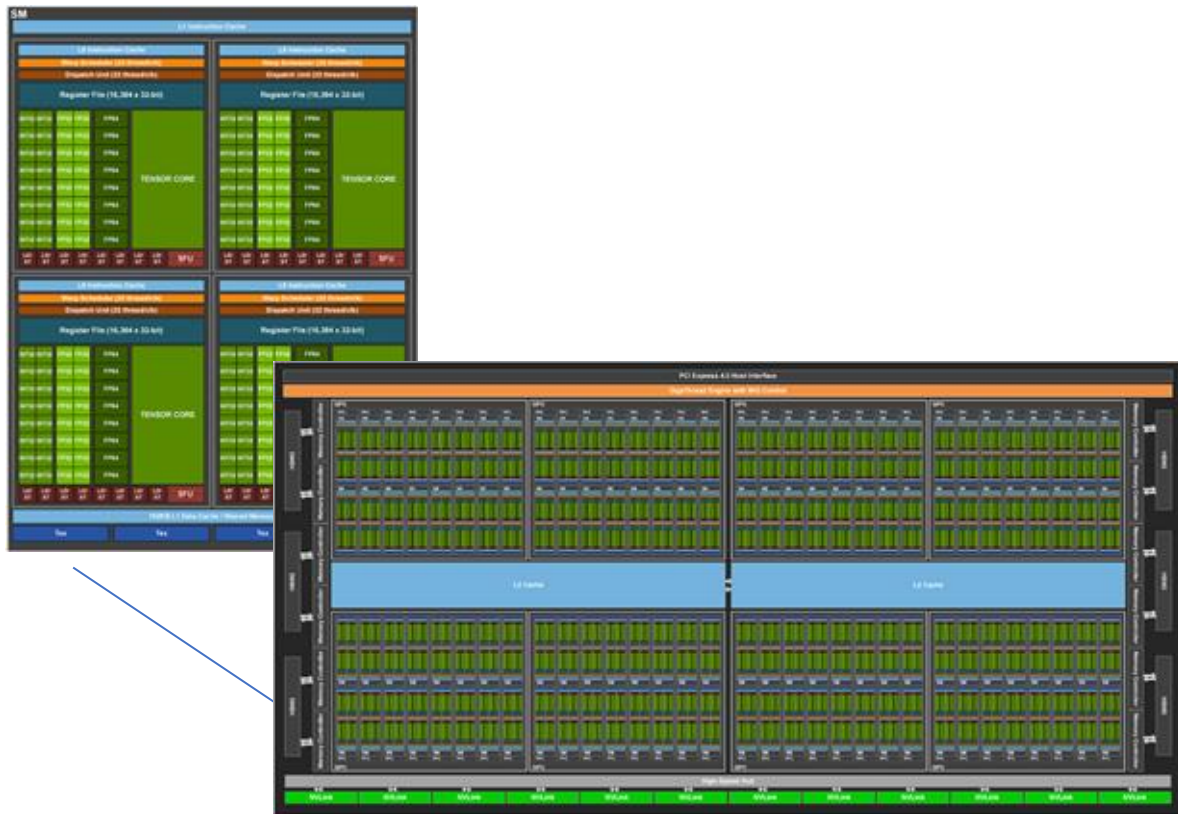
From parallel architectures to parallel programming models



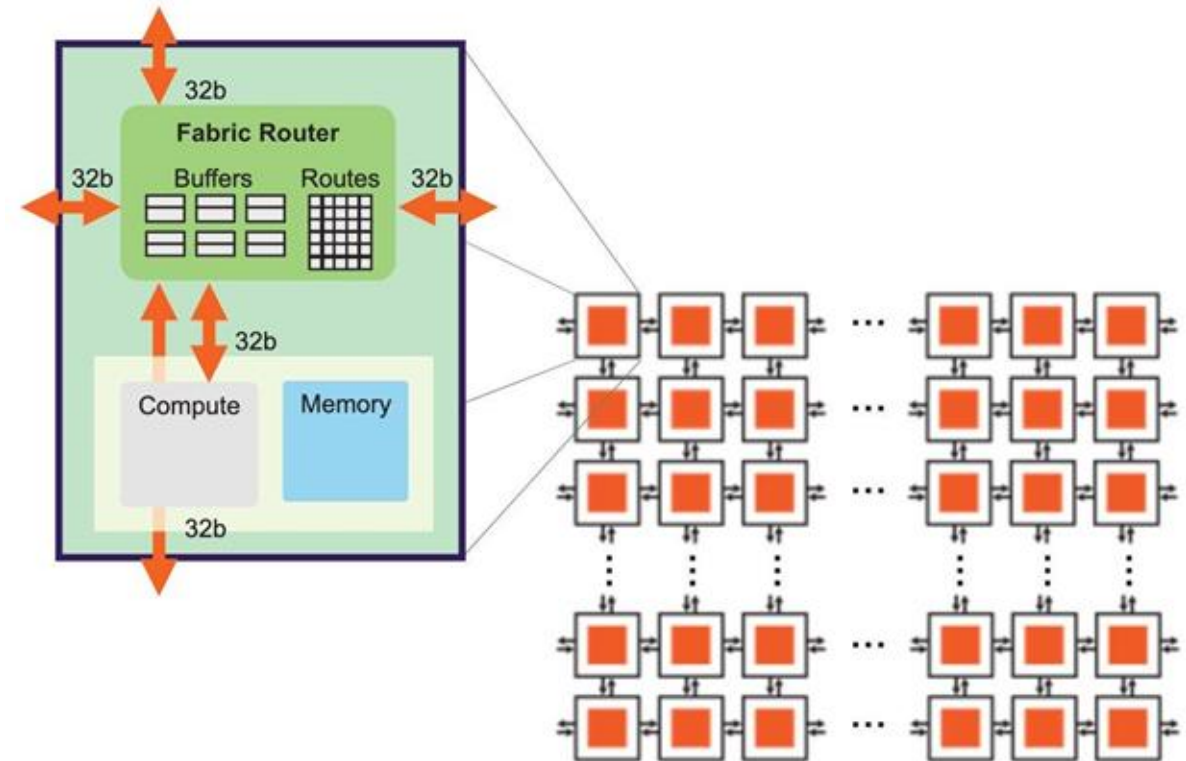
UNITRENTO

Modern parallel architectures

- **Accelerators** and domain-specific architectures (e.g., Graphics Processing Units, Tensor Processing Unit, Dataflow architectures)



NVIDIA Ampere A100



Cerebras' **second-generation Wafer-Scale architecture: the WSE-2 inside the CS-2 system,**

From parallel architectures to parallel programming models



UNITRENTO

All of them are parallel architectures

What are the main differences you see?

How to program them?

Parallel computing from software level

We can consider an abstract parallel machine

- Programming models allow to expose features at a higher level of abstraction (e.g., computational unit, memory)
- Ignore the implementation of a specific set of features (these features can be implemented in hardware or with efficient algorithms)

From this perspective we need to consider the dualism software / hardware

Examples

- Data type vs physical representation of the words in the memory

In C, you declare an integer without knowing the specific representation (for example little or big endian)

- Threads and Processes vs Physical Cores
- Barriers and synchronization mechanism vs Memory Transactions and Atomic operations

Different abstractions

Accelerated Standard Languages

```
std::transform(par, x, x+n, y, y,  
    [=](float x, float y){ return y + a*x;  
});
```

```
do concurrent (i = 1:n)  
    y(i) = y(i) + a*x(i)  
enddo
```

```
import legate.numpy as np  
...  
def saxpy(a, x, y):  
    y[:] += a*x
```

Incremental Portable Optimization

```
#pragma acc data copy(x,y) {  
    ...  
#pragma acc parallel loop  
for (i=0; i<n; i++) {  
    y[i] += a * x[i];  
}  
...  
}
```

```
#pragma omp target data map(x,y) {  
    ...  
#pragma omp target teams loop  
for (i=0; i<n; i++) {  
    y[i] += a * x[i];  
}  
...  
}
```

Platform Specialization

```
__global__  
void saxpy(int n, float a,  
    float *x, float *y) {  
    int i = blockIdx.x*blockDim.x +  
        threadIdx.x;  
    if (i < n) y[i] += a*x[i];  
}  
  
int main(void) {  
    ...  
    cudaMemcpy(d_x, x, ...);  
    cudaMemcpy(d_y, y, ...);  
  
    saxpy<<<(N+255)/256,256>>>(...);  
  
    cudaMemcpy(y, d_y, ...);
```

Performance Considerations

Parallel computing and performance

Performance is the measure of the completion of a task according to a specific measure.

- Metrics are, e.g., time, power consumption, resource utilizations etc..
- Theoretical computational models

Composed metrics: operation per time unit (instructions, floating point operations, memory access), efficiency, latency, scalability (weak and strong).

Domain specific units (e.g., inference per time unit, edges per time unit..).

Parallel computing and performance

Portability is the capacity of software to run on different architectures or environments.

To this aim for example, OpenCL provides a programming model and level of abstraction to match different architectures.

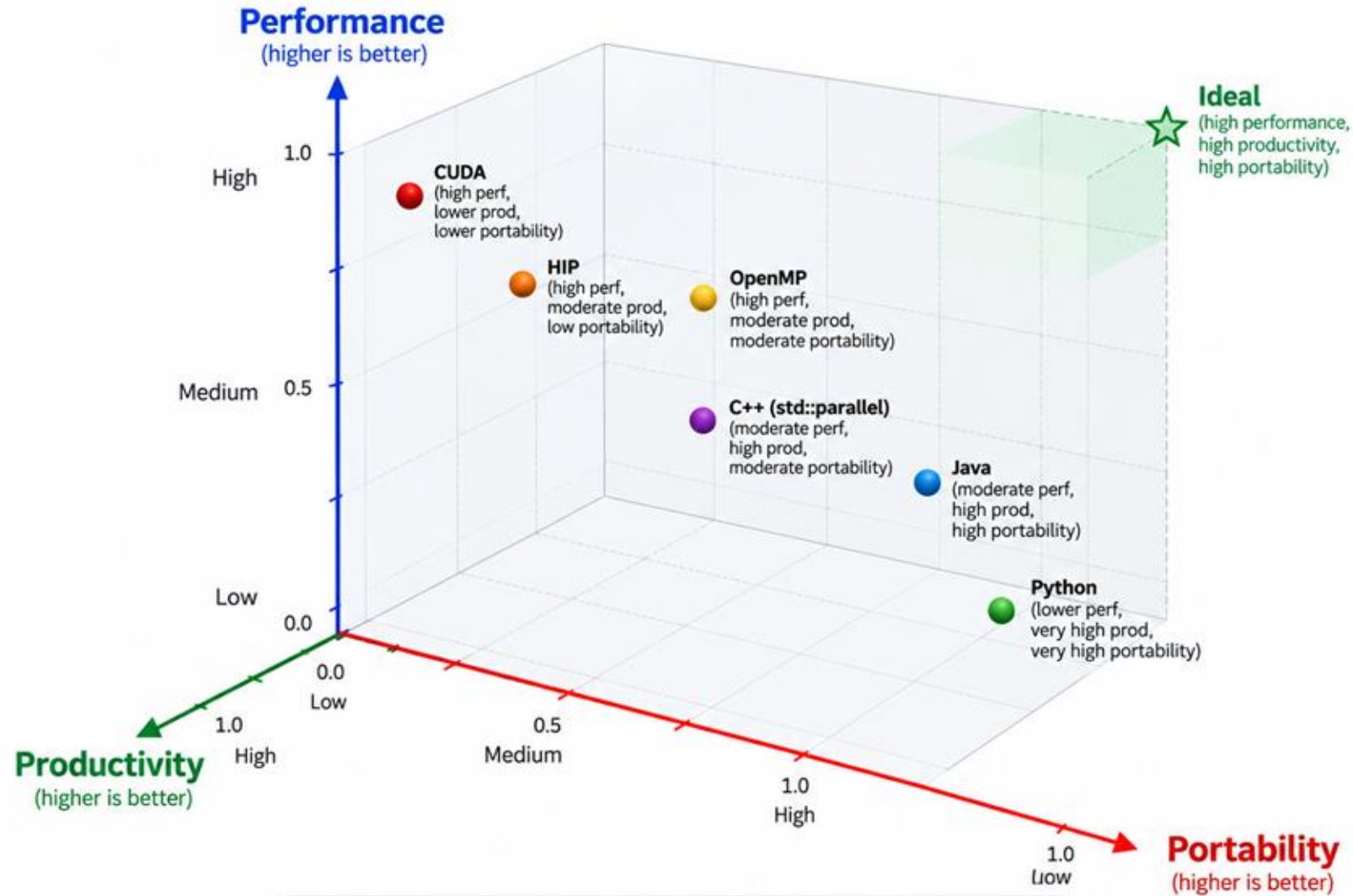
Performance portability: is the ability of the software to achieve a “consistent” level of performance across multiple architectures.

Productivity reflects the effort required to develop, debug, tune, and maintain parallel software.

- Productivity via parallel programming models (add more abstractions into a standard)
- Productivity via domain-specific programming models (e.g., identification algorithm primitives)

Performance – Productivity – Portability Trade-off

Different programming models make different trade-offs



Key message:

Increasing one dimension often comes at the expense of the others.
The best choice depends on your application and constraints.

Takeaway



UNITRENTO

1. Technology limits have made parallelism essential.
2. Parallel architectures are entering a new golden age.
3. Algorithms must be redesigned to expose parallelism.
4. We need to understand how to decompose them and coordinating task
5. HPC enables problems to be solved at unprecedented scale.
6. Scalability depends on communication, synchronization, and data locality.

Supplement Material



UNITRENT

- “A New Golden Age for Computer Architecture: Domain-Specific Hardware/Software Co-Design, Enhanced Security, Open Instruction Sets, and Agile Chip Development”, <https://www.acm.org/hennessy-patterson-turing-lecture>
- “Cramming more components onto integrated circuits” by Gordon Moore
- Productivity in Parallel Programming: A Decade of Progress, John T. Richards et al. (ACM queue)

References



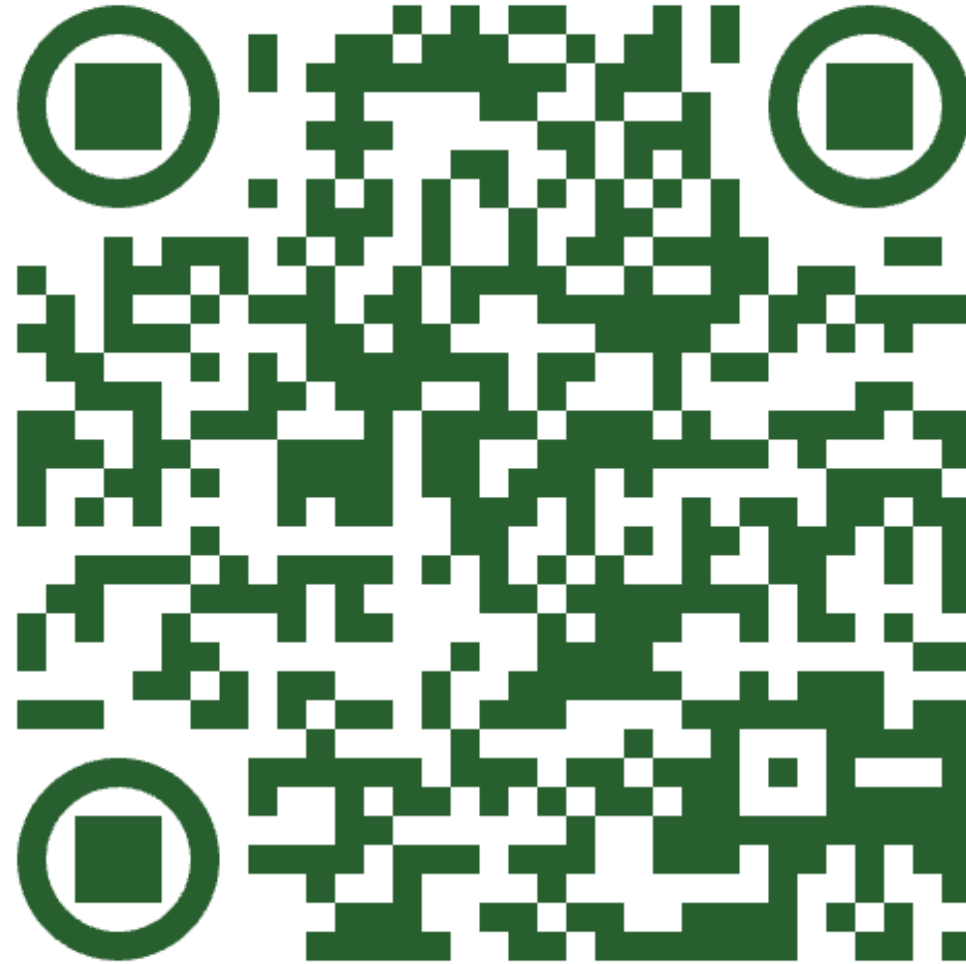
UNITRENTO

- Parallel and High-Performance Computing (Zamora). Chapter 1 “Why parallel computing?”
- An Introduction to Parallel Programming (Pacheco). Chapter 1 “Why parallel computing?” sections 1.1 – 1.2 – 1.3. – 1.4

Course Registration



UNITRENTO



<https://forms.gle/EGiph1sNroMbRnTm8>

Feedback Form

Constructive feedback is always welcome

**Do not wait until the end of the course to
just complain**

**Open the browser in incognito mode
The form is anonymous**

Please use it



<https://forms.gle/8bS9tvxgAiBQyr3s8>

Questions?